

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ "ОДЕСЬКА ПОЛІТЕХНІКА"

Інститут комп'ютерних наук
Кафедра прикладної математики

Сергій СОЦЬКИЙ
(група АВ221)

КВАЛІФІКАЦІЙНА РОБОТА БАКАЛАВРА
Creating a service-oriented architecture for scalable use
of language models in commercial solutions

Спеціальність 113 Прикладна математика
Освітньо-професійна програма "Прикладна математика"

Керівник: Грішина Віра Олександрівна

Одеса - 2026

АНОТАЦІЯ

Соцький С. Creating a service-oriented architecture for scalable use of language models in commercial solutions: кваліфікаційна робота бакалавра зі спеціальності 113 Прикладна математика / Сергій Соцький; керівник Віра Грішина. Одеса: Національний університет "Одеська політехніка", 2026.

У роботі розроблено, формалізовано, реалізовано та перевірено ModelWeave - сервісно-орієнтовану платформу документно обґрунтованих агентів для масштабованого використання мовних моделей у комерційних рішеннях. Система дає користувачеві змогу реєструватися, додавати власний ключ Anthropic, завантажувати PDF, TXT, MD і DOCX до бази знань, створювати агентів, будувати графи процесів, запускати виконання вручну або за розкладом, переглядати затвердження, перетворювати прийняті пропозиції на задачі та експортувати DOCX-звіти.

Математична частина описує корпус документів, фрагментацію, ембедінги, нормалізацію векторів, косинусну подібність, top-k пошук, RAG-генерацію, граф процесу, переходи станів, чергу затверджень, функцію оцінювання якості, моделі вартості, затримки та складності. Практична частина демонструє повностекову реалізацію на Next.js, FastAPI, Supabase, Qdrant Cloud, Anthropic API, Vercel і Railway.

Ключові слова: сервісно-орієнтована архітектура, мовні моделі, RAG, ембедінги, векторний пошук, мультиагентні системи, граф процесу, затвердження, Supabase, Qdrant, FastAPI, Next.js, DOCX.

ABSTRACT

Sotskyi S. Creating a service-oriented architecture for scalable use of language models in commercial solutions: bachelor qualification thesis in Applied Mathematics / Serhii Sotskyi; supervisor Vira Hrishyna. Odesa: Odesa Polytechnic National University, 2026.

The thesis presents the design, mathematical formalization, implementation, deployment, and evaluation of ModelWeave, a service-oriented platform for document-grounded language-model agents. The prototype supports user authentication, bring-

your-own Anthropic keys, document ingestion, vector retrieval, editable agents, visual workflows, manual and scheduled runs, approval gates, durable tasks, run evaluation, audit events, and DOCX report export.

The mathematical chapter formalizes document chunking, embedding vectors, cosine similarity, top-k retrieval, retrieval-augmented generation, typed workflow graphs, state transitions, approval-gated mutation, quality scoring, latency, cost, and storage complexity. The implementation uses Next.js, FastAPI, Supabase, Qdrant Cloud, Anthropic API, Vercel, and Railway. Only synthetic academic data is used.

ЗМІСТ

Перелік умовних позначень	6
Вступ	7
1 Теоретичні основи масштабованих систем мовних моделей	10
1.1 Мовні моделі як компоненти програмних систем	10
1.2 Сервісно-орієнтована архітектура	11
1.3 RAG, ембедінги та векторні бази даних	12
1.4 Мультиагентні процеси та human-in-the-loop управління	13
Висновки до розділу 1	14
2 Вимоги та предметна модель платформи ModelWeave	15
2.1 Проблемна область та межі MVP	15
2.2 Функціональні вимоги	16
2.3 Нефункціональні вимоги	17
2.4 Предметні сутності та користувацькі сценарії	17
Висновки до розділу 2	18
3 Математична модель пошуку, агентів і виконання	20
3.1 Корпус, фрагментація та ембедінги	20
3.2 Подібність, ранжування та RAG-генерація	21
3.3 Граф процесу і модель агентів	23
3.4 Затвердження, оцінювання, вартість і складність	24
3.5 Деталізація параметрів масштабування і якості	26
3.6 Інваріанти коректності виконання	29
Висновки до розділу 3	31
4 Архітектура та реалізація ModelWeave	33
4.1 Загальна сервісна схема	33
4.2 Бекенд, сховища та API	34
4.3 Фронтенд і користувацький досвід	35
4.4 Розгортання	39
Висновки до розділу 4	40
5 Тестування, оцінювання та практична перевірка	41

5.1 Методика тестування	41
5.2 Результати перевірок	42
5.3 Обмеження та напрями розвитку	47
5.4 Оцінка готовності до демонстрації та продукційного використання	48
Висновки до розділу 5	50
Загальні висновки	51
Список використаних джерел	53
Додатки	55

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ

Позначення	Значення
AI	Штучний інтелект
ANN	Наближений пошук найближчих сусідів
API	Інтерфейс прикладного програмування
BYOK	Модель використання власного ключа провайдера
DOCX	Формат Word-документа Office Open XML
HNSW	Ієрархічний граф навігації малих світів для ANN-пошуку
LLM	Велика мовна модель
MVP	Мінімально життєздатний продукт
RAG	Генерація з доповненням пошуком
RLS	Безпека на рівні рядків
SOA	Сервісно-орієнтована архітектура
UI	Користувацький інтерфейс

ВСТУП

Актуальність теми зумовлена тим, що великі мовні моделі швидко переходять із експериментальних чат-інтерфейсів у комерційні інформаційні системи. У бізнес-середовищі модель не може існувати ізольовано: потрібні автентифікація, контроль доступу, сховище документів, пошук, аудит, керування станом, людські затвердження та формати результатів, які можна передавати користувачам.

Затверджена тема роботи сформульована англійською мовою: "Creating a service-oriented architecture for scalable use of language models in commercial solutions". У роботі ця тема реалізується через повностековий прототип ModelWeave, який демонструє, як мовна модель може працювати не як окремий чат, а як частина сервісно-орієнтованої платформи документно обґрунтованих агентів.

Проблема масштабування полягає не лише у збільшенні кількості запитів до моделі. Потрібно масштабувати документи, ролі агентів, графи процесів, черги затверджень, трасування, оцінювання якості та розгортання. Якщо хоча б один із цих аспектів залишається неформалізованим, система може генерувати правдоподібний текст, але не бути придатною для контрольованого використання.

Метою роботи є проєктування, математична формалізація, реалізація, розгортання та перевірка сервісно-орієнтованої архітектури, яка забезпечує масштабоване використання мовних моделей у комерційних рішеннях на прикладі платформи ModelWeave.

Для досягнення мети поставлено такі задачі: проаналізувати сучасні підходи до LLM, RAG, векторного пошуку та агентних систем; визначити вимоги до документно обґрунтованої платформи агентів; побудувати математичні моделі корпусу, ембедінгів, пошуку, RAG-генерації, графів процесів, затверджень та оцінювання; спроектувати сервісно-орієнтовану архітектуру; реалізувати вебзастосунок; виконати локальні та продукційні перевірки; підготувати доказові матеріали у вигляді скріншотів, таблиць, формул, тестів і DOCX-звітів.

Об'єктом дослідження є процес інтеграції великих мовних моделей у комерційні програмні рішення. Предметом дослідження є сервісно-орієнтована архітектура та математична модель платформи документно обґрунтованих агентів із векторним пошуком, графом процесів, затвердженнями та експортом звітів.

Методи дослідження включають системний аналіз, математичне моделювання, аналіз алгоритмів пошуку, проєктування архітектури, прототипування, API-тестування, візуальну перевірку UI та документів, а також експериментальну перевірку на синтетичному корпусі.

Наукова новизна полягає у формальному поєднанні RAG-пошуку, ролей агентів, типізованих вузлів процесу, затверджень і якості запусків в одну модель виконання. Практична цінність полягає у створенні розгорнутого прототипу, який можна перевіряти, демонструвати та розширювати.

У роботі використано лише синтетичні дані. Жодні реальні дані компаній, клієнтів або внутрішніх робочих процесів не застосовуються. Це важливо як з етичної точки зору, так і з погляду відтворюваності дипломного дослідження.

Продукційний фронтенд перевірявся за адресою <https://modelweave-two.vercel.app>. Продукційний бекенд доступний за адресою <https://api-production-e70a9.up.railway.app>. Публічний репозиторій розміщено за адресою <https://github.com/serhiiSotskyi/multi-agent-knowledge-platform>. Поточна продукційна перевірка підтвердила працездатність оновленого фронтенду та health endpoint бекенду; повне оновлення Railway-бекенду потребує повторної автентифікації Railway CLI.

Для постановки задачі ключовим є принцип: архітектурна надійність має розглядатися на тому самому рівні, що й якість промпта. У ModelWeave це реалізовано через розбиття системи на фронтенд, бекенд, автентифікацію, реляційне сховище, векторне сховище та LLM-провайдера. Без такого рішення демонстрація залишалася б набором непов'язаних промптів. Тому архітектурна частина роботи безпосередньо пов'язана з математичною моделлю та практичною перевіркою.

У контексті постановки задачі важливо, що математична модель потрібна для пояснення, чому RAG і агенти поводяться передбачувано. Прототип підтримує цю вимогу завдяки формалізації корпусу, векторів, top-k пошуку та графа виконання. Це зменшує ризик того, що якість системи оцінювалася б лише суб'єктивно, і робить систему придатною для академічного аналізу.

Ще один висновок для постановки задачі: результат має бути перевірним поза текстом роботи. Реалізація використовує публічний репозиторій, production links, screenshots, tests і DOCX artifacts; інакше дипломна робота описувала б задум, а не реалізований прототип. Таке рішення робить поведінку агентів не лише зручною, а й перевірною.

1 ТЕОРЕТИЧНІ ОСНОВИ МАСШТАБОВАНИХ СИСТЕМ МОВНИХ МОДЕЛЕЙ

1.1 Мовні моделі як компоненти програмних систем

Великі мовні моделі є ймовірнісними моделями послідовностей, які генерують наступні токени на основі попереднього контексту. Архітектура Transformer дала змогу масштабувати самоувагу та стала базою сучасних LLM [1]. Однак з інженерної точки зору LLM не є самостійним застосунком.

Комерційна система повинна знати, хто робить запит, які документи доступні користувачу, які дані можна передавати моделі, які дії дозволені без затвердження та де зберігається результат. Ці питання не вирішуються самим фактом наявності мовної моделі.

Модель може створити переконливий текст навіть тоді, коли контекст неповний або неактуальний. Тому для комерційних рішень важливо поєднувати генерацію з retrieval, цитуваннями, аудитом і контрольованими переходами стану.

У ModelWeave LLM використовується як сервісний компонент. Фронтенд не викликає провайдера напряду. Бекенд перевіряє користувача, отримує релевантні документи, формує рольовий промпт, викликає Anthropic API з ключем користувача та записує події запуску.

Такий підхід відповідає ідеї керованого AI workflow: мовна модель генерує текст і пропозиції, але не володіє всією системою. Власність стану залишається в базі даних, а право на сталі зміни проходить через затвердження.

Для LLM як компонента програмної системи ключовим є принцип: генеративна здатність не дорівнює операційній відповідальності. У ModelWeave це реалізовано через розміщення викликів Anthropic у бекенд-сервісі. Без такого рішення користувач отримувач би текст без контрольованого походження. Тому архітектурна частина роботи безпосередньо пов'язана з математичною моделлю та практичною перевіркою.

У контексті LLM як компонента програмної системи важливо, що доказовість має бути частиною UX. Прототип підтримує цю вимогу завдяки збереження citations, trace і event timeline. Це зменшує ризик того, що результат не

можна було б пояснити на захисті або в реальній команді, і робить систему придатною для академічного аналізу.

Ще один висновок для LLM як компонента програмної системи: ключ провайдера є персональною відповідальністю користувача. Реалізація використовує ВУОК-модель із зашифрованим зберіганням ключа; інакше прототип перетворився б на спільний неконтрольований проксі. Таке рішення робить поведінку агентів не лише зручною, а й перевірною.

1.2 Сервісно-орієнтована архітектура

Сервісно-орієнтована архітектура декомponує систему на сервіси з окремими відповідальностями та інтерфейсами. Для AI-рішень це особливо важливо, тому що різні частини мають різні режими відмови: автентифікація, база даних, векторний пошук, LLM API, генерація DOCX і фронтенд-побудова стану.

SOA у контексті LLM не означає лише мікросервіси. Головне - явні межі: де зберігаються документи, де формується embedding, де виконується пошук, де розміщується промптинг, де фіксується рішення людини та де формується фінальний документ.

ModelWeave використовує Next.js як клієнтський сервіс, FastAPI як API-шар, Supabase Auth як сервіс ідентичності, Supabase Postgres як реляційне сховище, Qdrant Cloud як векторну базу, Anthropic API як LLM-провайдера, Vercel і Railway як інфраструктуру розгортання.

Розділення сервісів полегшує масштабування. Векторний пошук може масштабуватися окремо від UI, а реляційні записи запусків не змішуються з embedding-векторами. Це зменшує ризик того, що зміна одного шару порушить усю систему.

Для дипломної роботи важливо, що кожен сервіс має перевірний доказ: health endpoint бекенду, production frontend, Supabase Auth flow, Qdrant retrieval, таблиці Postgres, action events, DOCX export і збережені скриншоти.

Таблиця 1.1 - Сервісна декомпозиція платформи

Сервіс	Відповідальність	Причина відокремлення
Next.js	Користувачський інтерфейс	UI має змінюватися без зміни логіки retrieval.

FastAPI	Захищені API та оркестрація	Бекенд контролює доступ, ключі та виконання.
Supabase Auth	Реєстрація і JWT	Ідентичність користувача є окремим сервісом.
Supabase Postgres	Сталі записи	Документи, агенти, задачі та запуски потребують транзакційності.
Qdrant	Векторний пошук	ANN-пошук має іншу модель даних і масштабування.
Anthropic API	Генерація LLM	Модель є зовнішнім провайдером із власною вартістю та SLA.

1.3 RAG, ембедінги та векторні бази даних

Retrieval-Augmented Generation поєднує пошук у зовнішньому корпусі з генерацією відповіді [2]. У такій схемі модель не покладається лише на параметричну пам'ять, а отримує релевантні фрагменти документів як частину контексту.

Ембедінги перетворюють текстові фрагменти на вектори в евклідовому просторі. Якщо запит і документні фрагменти кодуються однаковою функцією, семантично близькі тексти мають близькі векторні представлення. Це дає змогу ранжувати документи за змістом, а не лише за ключовими словами.

Векторні бази даних, зокрема Qdrant, зберігають вектори разом із payload-метаданими: ідентифікатором користувача, іменем файлу, номером фрагмента та типом документа. Ці метадані критичні для ізоляції користувачів і цитування.

RAG не усуває всі ризики. Якщо retrieval повертає нерелевантні фрагменти, модель може зробити помилковий висновок. Тому ModelWeave зберігає цитування, а оцінка запуску враховує citation coverage.

У системі підтримуються PDF, TXT, MD і DOCX. Це важливо, оскільки комерційні документи рідко мають один формат. Парсер приводить різні джерела до тексту, chunker створює одиниці пошуку, а векторний шар робить їх доступними для агентів.

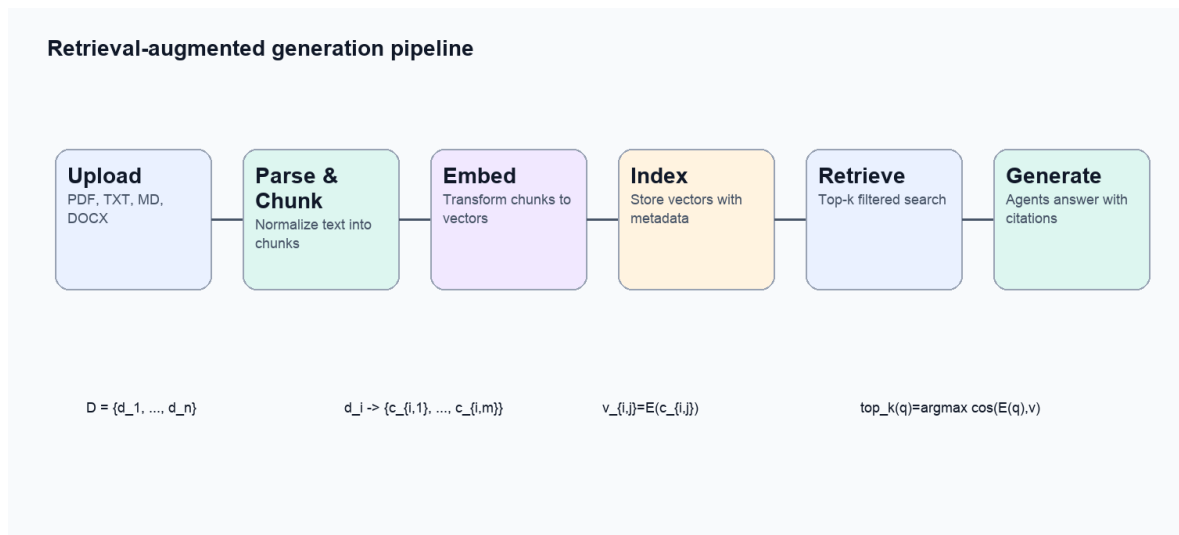


Рисунок 1.1 - Узагальнений RAG-процес у ModelWeave

1.4 Мультиагентні процеси та human-in-the-loop управління

Мультиагентний підхід розділяє складну задачу між спеціалізованими ролями. У ModelWeave користувач може створювати агентів природною мовою, задавати опис, системні інструкції, інструменти, режим запуску, статус і політику дозволів.

Агенти не повинні мати необмежених прав. Вони можуть формувати рекомендації та задачі, але сталі зміни мають проходити через approval gate. Такий human-in-the-loop механізм поєднує продуктивність LLM із людською відповідальністю.

Виконання процесів представлено графом. Вузли retrieve, agent, create_task, approval, evaluate і export_docx мають різні ролі. Це робить систему зрозумілою для користувача і зручною для тестування.

Успішні AI-продукти все частіше розглядають агентів як робочу силу або workflow layer, а не як один чат [27], [28], [29]. ModelWeave демонструє академічну версію такого підходу для документно обґрунтованих процесів.

Людське затвердження не є лише UI-кнопкою. Це формальна межа між пропозицією та мутацією стану. Відхилена пропозиція залишається в аудиті, але не стає затвердженою задачею.

Висновки до розділу 1

У розділі 1 встановлено теоретичну основу роботи. Масштабоване використання мовних моделей потребує сервісно-орієнтованої архітектури, retrieval, векторного пошуку, мультиагентних ролей, людських затверджень і трасування. Самостійний чатбот не відповідає цим вимогам, тому ModelWeave розглядає LLM як один із сервісів у повній програмній системі.

2 ВИМОГИ ТА ПРЕДМЕТНА МОДЕЛЬ ПЛАТФОРМИ MODELWEAVE

2.1 Проблемна область та межі MVP

Практична проблема полягає у створенні платформи, де користувач може працювати з власною документною базою, створювати агентів, будувати процеси, запускати їх вручну або за розкладом і отримувати перевірені результати. Така постановка є ширшою за звичайний RAG-чат, тому що результатом є не лише відповідь, а й робочий процес.

У поточній версії ModelWeave user-facing продукт зосереджений на семи поверхнях: Database, Agents, Workflows, Runs, Approvals, Tasks і Reports. Це спрощує демонстрацію та узгоджує її з темою сервісно-орієнтованої архітектури.

Database відповідає за документи. Agents відповідають за повторно використовуваних LLM-працівників. Workflows відповідають за графи виконання. Runs зберігають історію та live status. Approvals зберігають людські рішення. Tasks є затвердженими або запропонованими робочими одиницями. Reports забезпечують DOCX-вихід.

MVP свідомо не містить зовнішніх інтеграцій із рекламними, CRM або месенджерними сервісами. Це зменшує ризики й дозволяє зосередитися на математичній та архітектурній частині: RAG, графи, агенти, затвердження, оцінювання та документи.

Усі демонстраційні дані синтетичні. Це забезпечує академічну безпеку: роботу можна показувати, тестувати й публікувати без витоку реальних бізнес-даних.

Таблиця 2.1 - Поточна предметна модель UI

Поверхня UI	Основна функція	Архітектурний ресурс
Database	Завантаження, перегляд, редагування метаданих і видалення документів	documents, document_chunks, Qdrant vectors
Agents	Створення, налаштування, запуск і планування агентів	agents, runs
Workflows	Візуальне складання графів виконання	workflows.nodes, workflows.edges
Runs	Історія, live status, trace, citations, evaluation	runs, action_events, run_evaluations
Approvals	Людські рішення щодо	approvals

	пропозицій	
Tasks	Довготривалі робочі одиниці	agency_tasks
Reports	Перегляд і DOCX-експорт	runs, reports service

2.2 Функціональні вимоги

Користувач повинен мати змогу зареєструватися та увійти через реальний механізм автентифікації. У ModelWeave це реалізовано через Supabase Auth. Відкриту реєстрацію поєднано з вимогою BYOK: агентні запуски доступні лише після додавання користувацького ключа Anthropic.

Документна база повинна підтримувати кілька форматів: PDF, TXT, Markdown і DOCX. Після завантаження документ має бути прочитаний, розбитий на фрагменти, збережений у Postgres, проіндексований у Qdrant і доступний для відкриття в UI.

Конструктор агентів має підтримувати plain-English створення: користувач описує, що має робити агент, а система зберігає назву, роль, мету, опис, системні інструкції, інструменти, trigger type, trigger config, status і permission mode.

Візуальний конструктор процесів має дозволяти комбінувати вузли retrieve, agent, create_task, approval, evaluate і export_docx. Важливо, що вузол approval відокремлює генерацію пропозицій від сталих змін.

Запуски мають стартувати одразу. Run може коротко бути queued, але має швидко перейти в running. Він може завершитися completed або зупинитися у waiting_approval, якщо створено пропозиції, що потребують рішення людини.

Reports мають експортуватися у DOCX, а не залишатися raw Markdown. Це потрібно для академічної демонстрації та для практичної придатності результату.

Таблиця 2.2 - Функціональні вимоги та QA-докази

Вимога	Реалізація	Перевірка
Реальна автентифікація	Supabase Auth	Тимчасовий користувач у smoke test.
BYOK	Зашифроване зберігання Anthropic key	Provider key status configured.
PDF/TXT/MD/DOCX	Document parser + chunker	Upload/open/delete tests.
RAG	Qdrant retrieval	8 citations у clean run.
Агенти	Agents API + UI detail	Створення й run now.
Процеси	React Flow + backend graph execution	Workflow run events.

Затвердження	Approvals queue	3 pending approvals у тесті.
DOCX	Reports service	DOCX download > 40 KB у smoke test.

2.3 Нефункціональні вимоги

Безпека вимагає ізоляції користувачів. Кожен API-запит перевіряється через bearer token, а пошук у Qdrant фільтрується за user_id. Це означає, що навіть спільна векторна колекція не повинна змішувати документи різних користувачів.

Надійність вимагає журналу дій. Користувач має бачити, який вузол запущено, який агент відповів, коли створено approval і чи був доступний DOCX. Для цього ModelWeave зберігає action_events і current_node_label.

Продуктивність вимагає поділу швидких UI-операцій і повільних LLM-викликів. POST /api/runs повертає run_id одразу, а frontend опитує GET /api/runs/{id}. Така polling-модель простіша за WebSocket і достатня для академічного демо.

Супроводжуваність вимагає, щоб нові agent tools або workflow nodes додавалися без переписування всієї системи. Типізовані вузли дозволяють розширювати граф виконання поступово.

Якість документів вимагає, щоб Markdown-вихід агентів рендерився у вебзастосунку і перетворювався на справжні стилі Word у DOCX. Інакше користувач бачив би технічні символи розмітки, позначення жирного тексту або службові рядки markdown-таблиць.

2.4 Предметні сутності та користувацькі сценарії

Основними сутностями є document, document_chunk, agent, workflow, run, approval, task, action_event і run_evaluation. Ці сутності формують мінімальну предметну модель документно обґрунтованої агентної платформи.

Document описує файл. Document_chunk описує одиницю retrieval. Agent описує роль мовної моделі. Workflow описує граф виконання. Run описує конкретну спробу виконання. Approval описує людське рішення. Task описує сталу робочу дію. Action_event описує журнал виконання. Run_evaluation описує якість результату.

Типовий сценарій починається з реєстрації та додавання ключа Anthropic. Потім користувач завантажує документи або завантажує synthetic corpus. Далі він створює агентів або використовує default workforce, формує workflow, запускає його, перевіряє output, приймає або відхиляє approvals, переглядає tasks і завантажує DOCX-звіт.

Ручні запуски потрібні для демонстрації та разових задач. Scheduled runs потрібні для періодичних перевірок: наприклад, агент може регулярно переглядати базу документів і створювати задачі, якщо знаходить прогалини.

Різниця між Run, Approval і Task є принциповою. Run - це спроба виконання. Approval - це людське рішення перед зміною. Task - це довготривала робоча одиниця після пропозиції або затвердження.

Для предметної моделі ключовим є принцип: Run, Approval і Task не можна змішувати. У ModelWeave це реалізовано через окремі таблиці, екрани та API endpoint-и. Без такого рішення історія виконання втратила б відмінність між подією, рішенням і роботою. Тому архітектурна частина роботи безпосередньо пов'язана з математичною моделлю та практичною перевіркою.

У контексті предметної моделі важливо, що документи мають бути керованими після індексації. Прототип підтримує цю вимогу завдяки open, metadata edit і delete у Database. Це зменшує ризик того, що RAG міг би використовувати застарілі або небажані джерела, і робить систему придатною для академічного аналізу.

Ще один висновок для предметної моделі: scheduled execution має створювати такі самі run records, як manual execution. Реалізація використовує trigger_source='scheduled' у runs; інакше періодичні запуски були б невидимими у загальній історії. Таке рішення робить поведінку агентів не лише зручною, а й перевіркою.

Висновки до розділу 2

У розділі 2 визначено вимоги та предметну модель ModelWeave. Поточна система не обмежується звітами: вона містить базу документів, агентів, процеси, запуски, затвердження, задачі й DOCX-звіти. Функціональні та нефункціональні

вимоги на пряму відповідають архітектурі: автентифікація, BYOK, RAG, візуальний workflow, live status, human approval і document export утворюють цілісний продукт.

3 МАТЕМАТИЧНА МОДЕЛЬ ПОШУКУ, АГЕНТІВ І ВИКОНАННЯ

3.1 Корпус, фрагментація та ембедінги

Нехай D є скінченною множиною документів користувача. Кожен документ після парсингу поділяється на фрагменти. Фрагмент є мінімальною одиницею retrieval і цитування.

$$D = \{d_1, d_2, \dots, d_n\}$$

Формула 3.1 - Корпус документів користувача

$$d_i = \{c_{i,1}, c_{i,2}, \dots, c_{i,m_i}\}$$

Формула 3.2 - Документ як множина фрагментів

$$C = \bigcup_{i=1}^n d_i$$

Формула 3.3 - Загальна множина фрагментів

$$\varphi: C \rightarrow R^d, \quad \varphi(c_j) = e_j$$

Формула 3.4 - Функція ембедінгу

$$\hat{e}_j = \frac{e_j}{\|e_j\|_2}$$

Формула 3.5 - L2-нормалізація ембедінгу

Формула 3.1 визначає область пошуку для конкретного користувача. У багатокористувацькій системі глобальна база документів не є коректним корпусом для retrieval, тому що кожен користувач має власний набір документів і власні права доступу.

Фрагментація потрібна через обмеження контекстного вікна та вимоги точного цитування. Якщо передавати весь документ, модель отримає надлишковий контекст; якщо фрагмент занадто малий, він може втратити смислове оточення. Тому chunking є не лише технічною, а математичною операцією над корпусом.

Функція ембедінгу φ перетворює текст на числовий вектор. Розмірність d визначається embedding-моделлю. Вектор не є зрозумілим набором ручних ознак; це латентне представлення, придатність якого оцінюється якістю пошуку.

Нормалізація векторів дає змогу стабільніше порівнювати напрями у просторі. Для семантичного пошуку важливо, щоб рейтинг відображав змістову близькість, а не довжину вектора.

У ModelWeave фрагменти зберігаються у Postgres як текстові записи й у Qdrant як embedding-вектори. Таке дублювання навмисне: Postgres є джерелом структурованої істини, а Qdrant - індексом для семантичного пошуку.

3.2 Подібність, ранжування та RAG-генерація

Запит q також перетворюється на вектор. Далі система ранжує фрагменти за косинусною подібністю і вибирає top-k множину.

$$v_q = \phi(q)$$

Формула 3.6 - Ембедінг запиту

$$\dot{c}(q, c_j) = \cos(v_q, \hat{e}_j) = \frac{v_q \cdot \hat{e}_j}{\|v_q\|_2 \|\hat{e}_j\|_2}$$

Формула 3.7 - Косинусна подібність

$$R_k(q) = \arg \text{top-}k \ c_j \in C \ \sim(q, c_j)$$

Формула 3.8 - Тор-k множина retrieval

$$\text{context}(q) = \text{concat}(R_k(q), \text{metadata}, \text{citations})$$

Формула 3.9 - Побудова контексту

$$P(y \mid q, R_k(q), a) = \prod_{t=1}^T P(y_t \mid y_{<t}, q, R_k(q), a)$$

Формула 3.10 - RAG-генерація з роллю агента

$$\text{Precision}@k = \frac{|\text{relevant}(q) \cap R_k(q)|}{k}$$

Формула 3.11 - Точність retrieval

$$\text{Recall}@k = \frac{|\text{relevant}(q) \cap R_k(q)|}{|\text{relevant}(q)|}$$

Формула 3.12 - Повнота retrieval

$$\text{MRR} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\text{rank}(q)}$$

Формула 3.13 - Середній обернений ранг

Косинусна подібність оцінює кут між векторами. Якщо вектори нормалізовані, скалярний добуток напряму відповідає cosine score. Це зручно для векторних баз, які оптимізують пошук за схожістю.

Top-k retrieval є компромісом. Мале k може пропустити корисний документ, велике k збільшує контекст, вартість і ризик шуму. Для MVP $k=8$ є практичним значенням: воно дає достатньо доказів для цитувань і не перевантажує промпт.

Формула 3.10 показує, що відповідь агента залежить не лише від запиту й документів, а й від ролі а. Один і той самий retrieval-context може бути інтерпретований як звіт, перевірка ризиків або план задач залежно від agent prompt.

Precision@k і Recall@k є класичними метриками retrieval. У дипломному прототипі вони не обчислюються на великій розміченій вибірці, але математично пояснюють, чому citation coverage і completeness включено до evaluation score.

MRR важливий для інтерактивних систем, де користувач очікує, що найрелевантніше джерело буде у верхній частині списку. Якщо перший релевантний документ має низький rank, агент може побудувати відповідь на слабшому контексті.

RAG не гарантує істинність автоматично. Він лише додає зовнішні докази. Тому ModelWeave зберігає citations, trace і approval queue, щоб користувач міг перевірити, на чому базуються пропозиції.

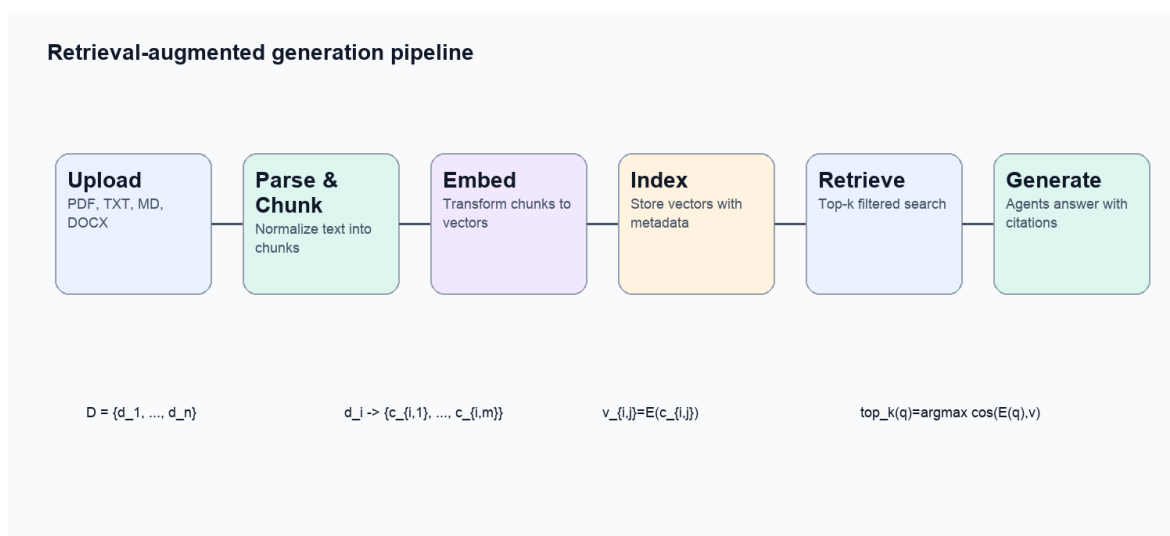


Рисунок 3.1 - Математична інтерпретація RAG-процесу

3.3 Граф процесу і модель агентів

Workflow у ModelWeave моделюється як орієнтований типізований граф. Вершини є вузлами виконання, ребра задають порядок, а функції типу й стану описують поведінку.

$$G = (V, E, \tau, \sigma)$$

Формула 3.14 - Типізований граф процесу

$$\tau(v) \in \{\text{retrieve}, \text{agent}, \text{create_task}, \text{approval}, \text{export_docx}\}$$

Формула 3.15 - Функція типу вузла

$$\sigma_t(v) \in \{\text{waiting}, \text{running}, \text{completed}, \text{approval_required}, \text{failed}\}$$

Формула 3.16 - Стан вузла

$$a_j = \langle \text{role}_j, \text{goal}_j, \text{tools}_j, \text{policy}_j \rangle$$

Формула 3.17 - Кортеж агента

$$x_{t+1} = F(x_t, v_t, a_j, R_k(q))$$

Формула 3.18 - Оновлення стану агентного вузла

$$\text{order}(G) = \text{topological_sort}(V, E)$$

Формула 3.19 - Порядок виконання графа

Формула 3.14 задає мінімальну модель workflow. Вона достатня для академічного MVP, тому що кожен вузол має тип і поточний стан. UI може відобразити цей стан, а бекенд може записати node_started, node_completed або node_failed.

Типи вузлів у Формулі 3.15 відповідають поточному спрощеному продукту. У ньому немає окремих користувацьких вузлів для зовнішніх інтеграцій; усі сталі дії зводяться до create_task і approval.

Кортеж агента відокремлює роль від моделі. Два агенти можуть використовувати той самий Anthropic model, але мати різні goals, tools і permission policy. Це дає змогу створювати спеціалізованих агентів без нової інфраструктури.

Topological sort потрібний для виконання графа без циклів. Якщо в майбутньому додати цикли або умовні переходи, модель доведеться розширити до станового автомата або workflow engine з умовами.

Живий RunProgressFlow у фронтенді є візуалізацією функції $\sigma_t(v)$. Коли вузол активний, він позначається running; коли створено approvals, запуск переходить у waiting_approval.

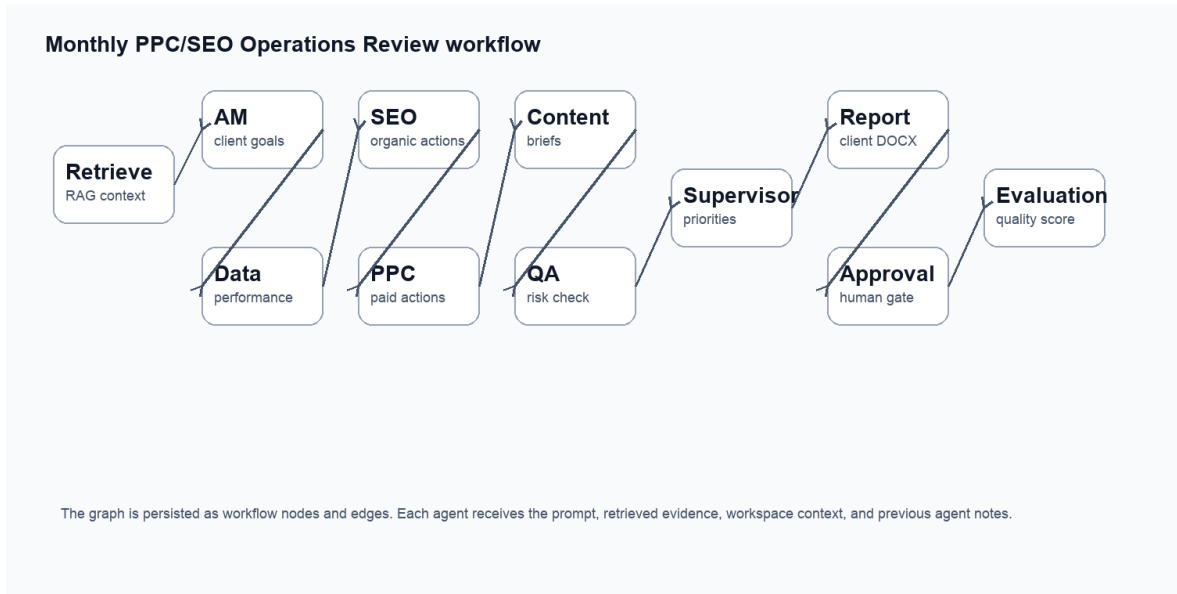


Рисунок 3.2 - Граф процесу з typed nodes

3.4 Затвердження, оцінювання, вартість і складність

Затвердження моделює межу між пропозицією та сталою зміною. Агент може створити запропоновану задачу, але остаточний стан визначає користувач.

$$\delta(s, \text{approved}) = s_{\text{approved}}, \quad \delta(s, \text{rejected}) = s_{\text{rejected}}$$

Формула 3.20 - Перехід рішення затвердження

$$A_{\text{pending}}(t) = \{a \mid \text{status}(a, t) = \text{pending_approval}\}$$

Формула 3.21 - Черга очікуваних затверджень

$$S = \frac{w_c C + w_a A + w_r R + w_m M}{w_c + w_a + w_r + w_m}$$

Формула 3.22 - Зважена оцінка якості

$$\text{Cost}(\text{run}) = \sum_{j=1}^m (p_{\text{in}} T_{i,j} + p_{\text{out}} T_{\text{out},j})$$

Формула 3.23 - Токенна модель вартості

$$\text{Latency}(\text{run}) \approx \sum_{v \in V} L_v + L_{\text{retrieval}} + L_{\text{export}}$$

Формула 3.24 - Модель затримки

$$\text{Storage}(C) = O(|C|d) + O(|C|m)$$

Формула 3.25 - Складність зберігання

$$T_{\text{exact}}(q) = O(|C|d), \quad T_{\text{ANN}}(q) \approx O(\log |C|)$$

Формула 3.26 - Точний і наближений пошук

$$C_u = \{c_j \in C \mid \text{owner}(c_j) = u\}$$

Формула 3.27 - Корпус, обмежений користувачем

$$N_{\text{calls}}(\text{run}) = |V_{\text{agent}}| + |V_{\text{evaluate}}| + |V_{\text{report}}|$$

Формула 3.28 - Кількість model-dependent викликів

Формула 3.20 показує, що approve і reject не є симетричними з погляду бізнес-результату, але обидва є сталими рішеннями в аудиті. Approved task може стати частиною робочої черги; rejected task не зникає безслідно, бо рішення теж є даними.

Формула 3.22 нормалізує якість запуску. С описує citation coverage, А - actionability, R - risk control, М - completeness. У MVP ваги рівні, але в майбутньому їх можна навчати або калібрувати за людськими оцінками.

Модель вартості у Формулі 3.23 пояснює, чому кількість агентів має значення. Multi-agent workflow може бути якіснішим, але дорожчим. Тому system design має контролювати кількість model calls і довжину контексту.

Формула 3.27 є математичним записом user isolation. Навіть якщо всі embeddings лежать в одній Qdrant collection, пошук конкретного користувача виконується у підмножині C_u .

Складність exact search росте лінійно з $|C|$ і d . ANN-структури на кшталт HNSW забезпечують значно практичніший час пошуку для великих корпусів [5].

Модель затримки важлива для UX. Якщо retrieval швидкий, але LLM-вузли повільні, Run screen має показувати прогрес, а не блокувати користувача порожнім станом.

Таблиця 3.1 - Математичні параметри системи

Компонент	Математичний параметр	Вплив на систему
Корпус	$ C $	Визначає обсяг векторного пошуку.
Embedding	d	Впливає на пам'ять і швидкість similarity.
Retrieval	k	Керує обсягом контексту і citation coverage.
Агенти	N_{calls}	Визначає кількість LLM-

		викликів.
Затвердження	A_pending(t)	Визначає навантаження на людину.
Оцінювання	S	Дає числовий summary якості запуску.



Рисунок 3.3 - Компоненти оцінювання запуску

3.5 Деталізація параметрів масштабування і якості

Для прикладної математики важливо не лише перелічити компоненти системи, а й показати, як параметри цих компонентів впливають на якість, швидкодію та вартість. У ModelWeave такими параметрами є розмір корпусу, середня довжина документа, розмір фрагмента, overlap між фрагментами, розмірність embedding-вектора, глибина top-k retrieval, кількість агентних вузлів і кількість approval gates.

$$m_i = \text{ceil}\left(\frac{L_i - o}{b - o}\right)$$

Формула 3.29 - Кількість фрагментів для документа з overlap

$$|C| = \sum_{i=1}^n m_i$$

Формула 3.30 - Загальна кількість фрагментів після chunking

$$M_vectors = |C| \cdot d \cdot s$$

Формула 3.31 - Орієнтовна пам'ять для векторів

$$B_context = k \cdot b + B_prompt + B_trace$$

Формула 3.32 - Оцінка довжини контексту агента

У Формулі 3.29 L_i позначає довжину документа в токенах або символах після нормалізації, b - розмір одного фрагмента, а o - overlap. Якщо overlap дорівнює нулю, модель отримує менше дублювання, але boundary-effect стає сильнішим. Якщо overlap занадто великий, retrieval отримує більше схожих фрагментів і витрачає пам'ять на майже повторюваний контекст.

Формула 3.30 показує, що кількість фрагментів росте не лише з кількістю документів, а й зі способом chunking. Два корпуси з однаковою кількістю файлів можуть мати різний retrieval-cost, якщо один містить довгі DOCX-документи, а інший короткі Markdown-нотатки.

Формула 3.31 описує пам'ять для збереження embedding-векторів. Параметр s означає кількість байтів на одну координату. Для float32 $s=4$, тому перехід від $d=384$ до $d=1536$ збільшує векторну пам'ять у чотири рази. Це пояснює, чому embedding-модель є архітектурним вибором, а не лише ML-деталлю.

Формула 3.32 показує, що контекст агента складається з retrieval-фрагментів, системного промпта і службової інформації trace. Якщо k або b збільшується, вхідна довжина prompt також збільшується. Це впливає на вартість, затримку і ймовірність того, що модель сфокусується не на головних доказах.

У прототипі практичне значення $k=8$ обране як компроміс між coverage і cost. Для фінальної демонстрації чистий запуск створив 8 citations, що відповідає цьому параметру. У більших системах k може адаптуватися до типу задачі: коротке питання потребує меншого k , а звіт із доказами - більшого k .

Таблиця 3.2 - Компроміси параметрів масштабування

Параметр	Збільшення параметра	Очікуваний ефект	Ризик
b	Більші фрагменти	Кращий локальний контекст	Шум і більша довжина prompt
o	Більший overlap	Менше втрат на межах	Дублікати та більше vectors
d	Вища розмірність	Багатше представлення	Більше пам'яті та повільніший exact search
k	Більше retrieved chunks	Вища recall	Вища вартість і більше шуму
$ V_{agent} $	Більше агентів	Більше спеціалізації	Більше LLM calls і довший run
$ A_{pending} $	Більше approvals	Більший контроль	Більше навантаження на

			користувача
--	--	--	-------------

Вибір параметрів не має універсально правильного значення. Він залежить від типу документів, очікуваної довжини відповіді, бюджету токенів і того, наскільки користувач готовий перевіряти пропозиції агентів. Тому архітектура повинна давати змогу змінювати параметри без переписування всього продукту.

Якщо розглядати систему як оптимізаційну задачу, можна поставити мету максимізувати якість S за обмежень на вартість і затримку. Це приводить до задачі умовної оптимізації, у якій змінними є k , кількість агентів, довжина контексту та кількість approval gates.

$$\text{maximize } S(k, |V_{\text{agent}}|, b, o)$$

Формула 3.33 - Ціль оптимізації якості

$$\text{subject to } \text{Cost}(\text{run}) \leq B, \quad \text{Latency}(\text{run}) \leq L_{\text{max}}$$

Формула 3.34 - Обмеження бюджету і затримки

Формули 3.33-3.34 не реалізовані як автоматичний оптимізатор у MVP, але вони пояснюють, як прототип може розвиватися. Наприклад, система може автоматично зменшувати k для коротких agent runs або підвищувати k для final report, якщо потрібне ширше цитування.

Другий напрям формалізації стосується якості evidence. Наявність цитування не гарантує, що цитата справді підтримує твердження. Тому для більш строгої системи можна ввести функцію $\text{support}(y_t, c_j)$, яка оцінює, чи підтримує фрагмент конкретний фрагмент відповіді.

$$\text{support}(y_t, c_j) \in [0,1]$$

Формула 3.35 - Підтримка твердження джерелом

$$\text{Grounding}(y) = \frac{1}{r} \sum_{l=1}^r \max_{c_j \in R_k} \text{support}(y_l, c_j)$$

Формула 3.36 - Узагальнена оцінка обґрунтованості

Формула 3.36 є розвитком поточної евристики citation coverage. Вона показує, що майбутня система може оцінювати не лише кількість цитувань, а й

відповідність тверджень джерелам. Це особливо важливо для звітів, де помилкове посилання може створити хибне враження доказовості.

Для agent workflows також важлива стійкість до помилки одного вузла. Якщо всі вузли виконуються послідовно, відмова одного LLM call блокує весь run. Тому production-grade версія повинна мати retry policy і idempotency key для операцій, які можуть повторюватися.

$$P_success(run) = \prod_{v \in V} P_success(v)$$

Формула 3.37 - Ймовірність успіху послідовного процесу

$$P_success_retry(v) = 1 - (1 - p_v)^r$$

Формула 3.38 - Успіх вузла з r повторними спробами

Формула 3.37 пояснює, чому довгі мультиагентні процеси потребують надійної інфраструктури. Навіть якщо кожен окремий вузол має високу ймовірність успіху, добуток багатьох імовірностей може бути помітно меншим. Це математичне пояснення потреби у durable queue, retries і trace.

Формула 3.38 показує позитивний ефект повторних спроб для тимчасових відмов. Якщо ймовірність успіху одного виклику дорівнює p_v , то r незалежних спроб підвищують шанси завершення вузла. Однак retries не повинні повторно створювати задачі або approvals, тому idempotency є частиною математичної коректності системи станів.

У ModelWeave MVP durable queue ще не використовується, але action_events і run.status уже формують основу для майбутньої надійної оркестрації. Вони зберігають достатньо інформації, щоб пояснити, де саме run зупинився або перейшов у waiting_approval.

3.6 Інваріанти коректності виконання

Коректність агентної системи можна описати через інваріанти. Інваріант - це твердження, яке має залишатися істинним після кожної операції. Для ModelWeave важливі інваріанти ізоляції, затверджень, аудитованості, цілісності задач і відтворюваності звіту.

$$\forall c \in result(u, q): owner(c) = u$$

Формула 3.39 - Інваріант ізоляції retrieval

$$mutate(x) \Rightarrow \exists a: approval(a, x) = approved$$

Формула 3.40 - Інваріант затвердження мутації

$$\forall event\ e: e.run_id \neq \emptyset \wedge e.user_id = owner(run)$$

Формула 3.41 - Інваріант журналу подій

$$report(run) = f(output, trace, citations, approvals, evaluation)$$

Формула 3.42 - Відтворюваність DOCX-звіту

Формула 3.39 є строгим записом того, що retrieval не повинен повертати фрагменти іншого користувача. У реалізації це підтримується фільтром user_id у векторному сховищі та перевіркою автентифікації у backend routes.

Формула 3.40 описує головний принцип human-in-the-loop: сталу зміну не можна виконати лише на підставі тексту агента. Потрібен запис approval зі статусом approved. Якщо рішення rejected, action залишається у журналі, але не змінює фінальний робочий стан.

Формула 3.41 забезпечує аудитованість. Кожна подія має бути пов'язана із запуском і користувачем. Без цього timeline у UI був би декоративним, а не доказовим об'єктом.

Формула 3.42 визначає DOCX-звіт як функцію збережених артефактів run. Це означає, що звіт не є випадковим новим текстом, згенерованим без зв'язку з виконанням; він має включати output, trace, citations, approvals і evaluation.

Ці інваріанти дозволяють перевіряти систему не лише функціональними тестами, а й логічними твердженнями. Наприклад, тест approvals перевіряє Формулу 3.40: rejected approval не повинен створювати approved task, а approved approval може перевести proposed task у робочий стан.

Інваріанти також пояснюють, чому старі доменні елементи були прибрані з UI. Якщо продукт позиціонується як загальна документно обґрунтована агентна платформа, його інваріанти мають стосуватися документів, агентів, запусків, затверджень і задач, а не спеціалізованої предметної області.

З погляду прикладної математики система є дискретною динамічною системою зі станом, переходами та обмеженнями. Стан складається з множин

documents, agents, workflows, runs, approvals, tasks і events. Операції API є переходами, які повинні зберігати інваріанти.

Таблиця 3.3 - Інваріанти коректності ModelWeave

Інваріант	Реалізаційний механізм	Тестовий прояв
Ізоляція retrieval	JWT + user_id filter у Qdrant	Користувач бачить лише власні citations.
Approval before mutation	approvals.status і protected task transition	Rejected не створює фінальну task.
Auditability	action_events	Runs screen показує timeline.
Run liveness	queued -> running background transition	Manual run не зависає queued.
Report reproducibility	DOCX із run artifacts	Report містить output, citations і evaluation.

Розгляд інваріантів допомагає підготувати тест-план. Замість того щоб перевіряти лише наявність кнопок у UI, тест має встановити, що натискання кнопки не порушує математичну модель стану. Саме тому smoke test перевіряв документи, агенти, запуски, approvals, tasks і DOCX як один сценарій.

У майбутніх версіях ці інваріанти можуть бути закріплені на рівні бази даних: foreign keys, row-level security, constraints і transactional operations. У MVP частина обмежень реалізована у backend service layer, що є прийнятним для академічного прототипу, але потребує посилення для production-grade платформи.

Таким чином, математична модель ModelWeave охоплює не лише embeddings і cosine similarity, а й повний життєвий цикл агентної дії. Це важливо для теми роботи, бо масштабоване використання LLM у комерційних рішеннях неможливе без формальної моделі даних, станів, переходів, витрат і ризиків.

Висновки до розділу 3

У розділі 3 побудовано математичну модель ModelWeave. Документи формалізовано як множину фрагментів, фрагменти - як вектори, retrieval - як top-k ранжування за косинусною подібністю, генерацію - як умовний процес з урахуванням ролі агента, workflow - як типізований граф, approval - як обмежений перехід стану, а quality score - як нормалізовану зважену функцію. Ця модель демонструє математичну основу RAG, vectorization, agent orchestration і human-in-the-loop управління.

4 АРХІТЕКТУРА ТА РЕАЛІЗАЦІЯ MODELWEAVE

4.1 Загальна сервісна схема

ModelWeave реалізовано як повностековий застосунок. Його архітектура розділяє клієнтську взаємодію, API, автентифікацію, реляційний стан, векторний індекс, LLM-виклики та документний експорт.

Користувач працює з браузерним UI. Усі захищені дії проходять через FastAPI. Бекенд перевіряє Supabase session, працює з Postgres, викликає Qdrant для retrieval, викликає Anthropic для генерації та формує DOCX.

Такий поділ відповідає темі роботи: scalable use of language models неможливий без архітектури навколо моделі. Модель є важливим, але не єдиним сервісом.

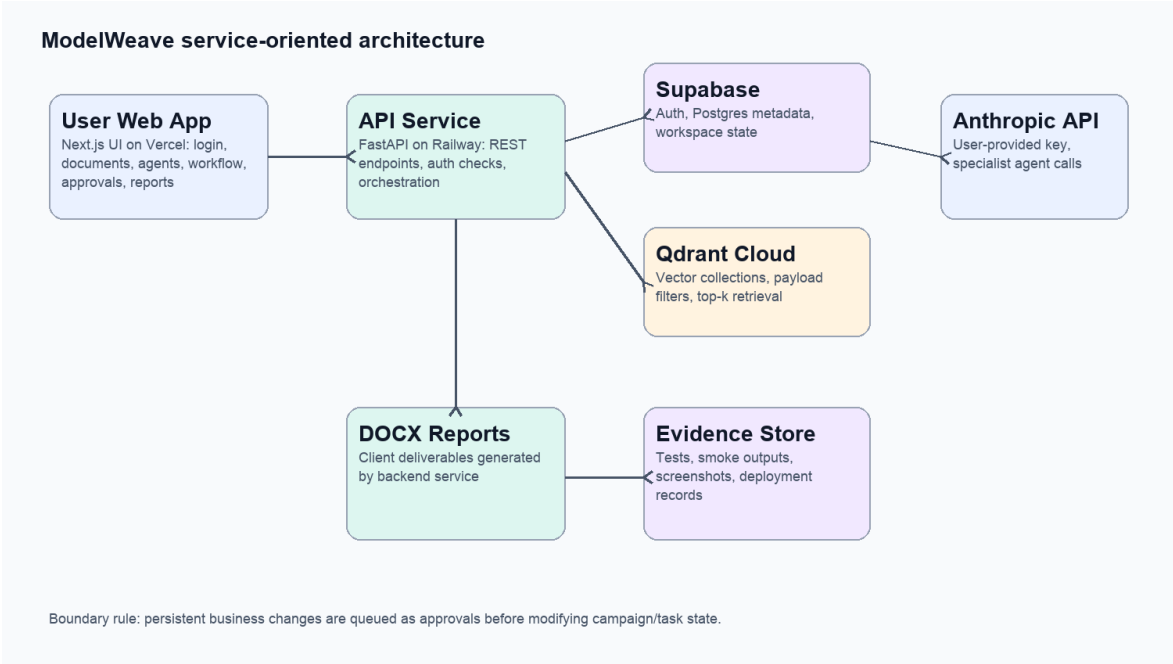


Рисунок 4.1 - Сервісно-орієнтована архітектура ModelWeave

Таблиця 4.1 - Реалізаційний стек

Шар	Технологія	Реалізаційна роль
UI	Next.js	Темний control plane, вкладки Database/Agents/Workflows/Runs/Approvals/Tasks/Reports.
Workflow UI	React Flow	Візуальний граф вузлів і live status.
Backend	FastAPI	Захищені маршрути, background execution, DOCX export.
Auth	Supabase Auth	Open registration і session validation.

Postgres	Supabase	Сталі сутності платформи.
Vector DB	Qdrant Cloud	Semantic retrieval over chunks.
LLM	Anthropic API	BYOK model calls for agents.

4.2 Бекенд, сховища та API

Бекенд організовано навколо API routes і сервісів. Routes відповідають за HTTP-контракти, services - за логіку агентів, документів, provider keys, reports і vector store. Це дозволяє тестувати бізнес-логіку незалежно від UI.

Міграції бази даних створюють таблиці documents, document_chunks, agents, workflows, runs, approvals, agency_tasks, action_events, run_evaluations і provider_keys. Додаткова міграція розширює agents полями description, system_prompt, trigger_type, trigger_config, status, permission_mode і last_scheduled_run_at.

Run execution створює запис queued, запускає background task і оновлює стан до running. Після вузла approval запуск переходить у waiting_approval, якщо є pending approvals. Це відповідає вимозі, щоб runs не залишалися queued без причини.

Scheduler endpoint створює scheduled runs для активних агентів із trigger_type='scheduled'. Усі scheduled runs записуються у ту саму таблицю runs і відрізняються trigger_source='scheduled'.

DOCX endpoint будує Word-документ із output, events, approvals, evaluation і citations. Це робить звіт частиною архітектури, а не зовнішнім ручним результатом.

Таблиця 4.2 - API-поверхня поточної системи

Endpoint group	Приклади	Призначення
Provider key	GET/PUT /api/provider-key	BYOK Anthropic configuration.
Database	GET/POST/PUT/DELETE /api/documents	Document ingestion and management.
Agents	GET/POST/PUT /api/agents; POST /api/agents/{id}/run	Agent builder and single-agent execution.
Scheduler	POST /api/agents/scheduler/run-due	Timed execution check.
Workflows	GET/POST/PUT /api/workflows	Visual graph persistence.
Runs	POST /api/runs; GET /api/runs/{id}	Manual execution and run detail.
Approvals	POST /api/approvals/{id}/approve reject	Human decision gate.

Reports	GET /api/runs/{id}/docx	DOCX export.
---------	-------------------------	--------------

4.3 Фронтенд і користувацький досвід

Фронтенд був оновлений до темного workspace control plane. Основна навігація містить лише актуальні продуктні поверхні. Це зменшує когнітивне навантаження та прибирає зайві доменні елементи.

Database screen показує документи як керовану базу знань. Користувач може завантажити файл, завантажити synthetic corpus, відкрити документ, побачити chunk count, статус індексації, дату й видалити документ.

Agents screen підтримує створення агентів у зрозумілій формі: опис задачі, trigger type, schedule text, status, permissions і detail view. Run now дає змогу протестувати агента без побудови повного workflow.

Workflows screen використовує React Flow для редагування графа. Вузли спрощені до retrieve, agent, create_task, approval, evaluate і export_docx. Це відповідає академічній цілі: показати алгоритм execution без зайвих зовнішніх інтеграцій.

Runs screen є єдиною історією виконання. Він показує manual і scheduled runs, live graph, event timeline, trace, citations, approvals, evaluation і DOCX action. Це усуває дублювання між різними екранами запусків.

Approvals і Tasks розділяють людське рішення й сталу роботу. Approval відповідає на питання «чи дозволити дію?», а Task відповідає на питання «що тепер потрібно зробити?».

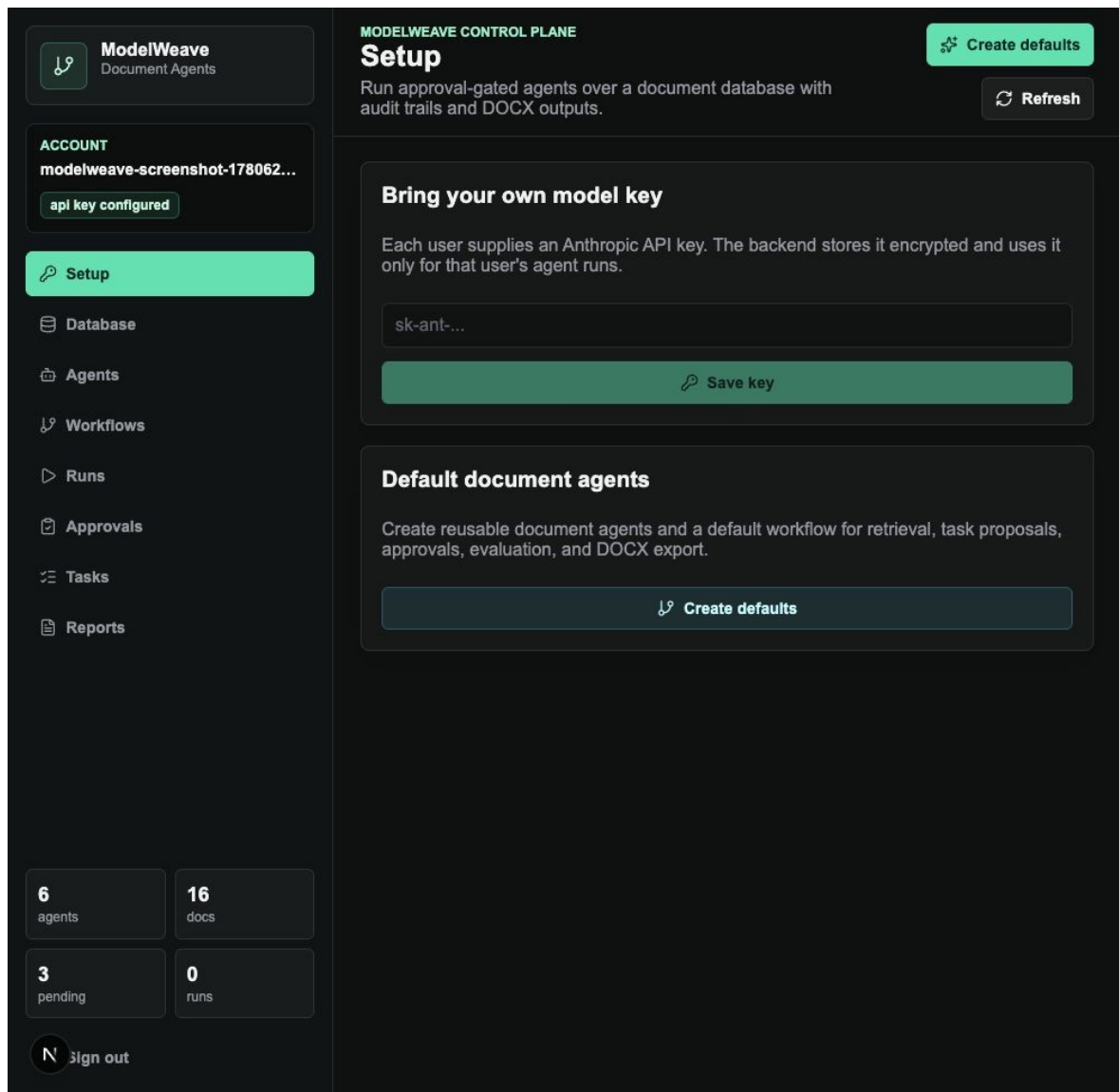


Рисунок 4.2 - Setup та ВУОК-налаштування

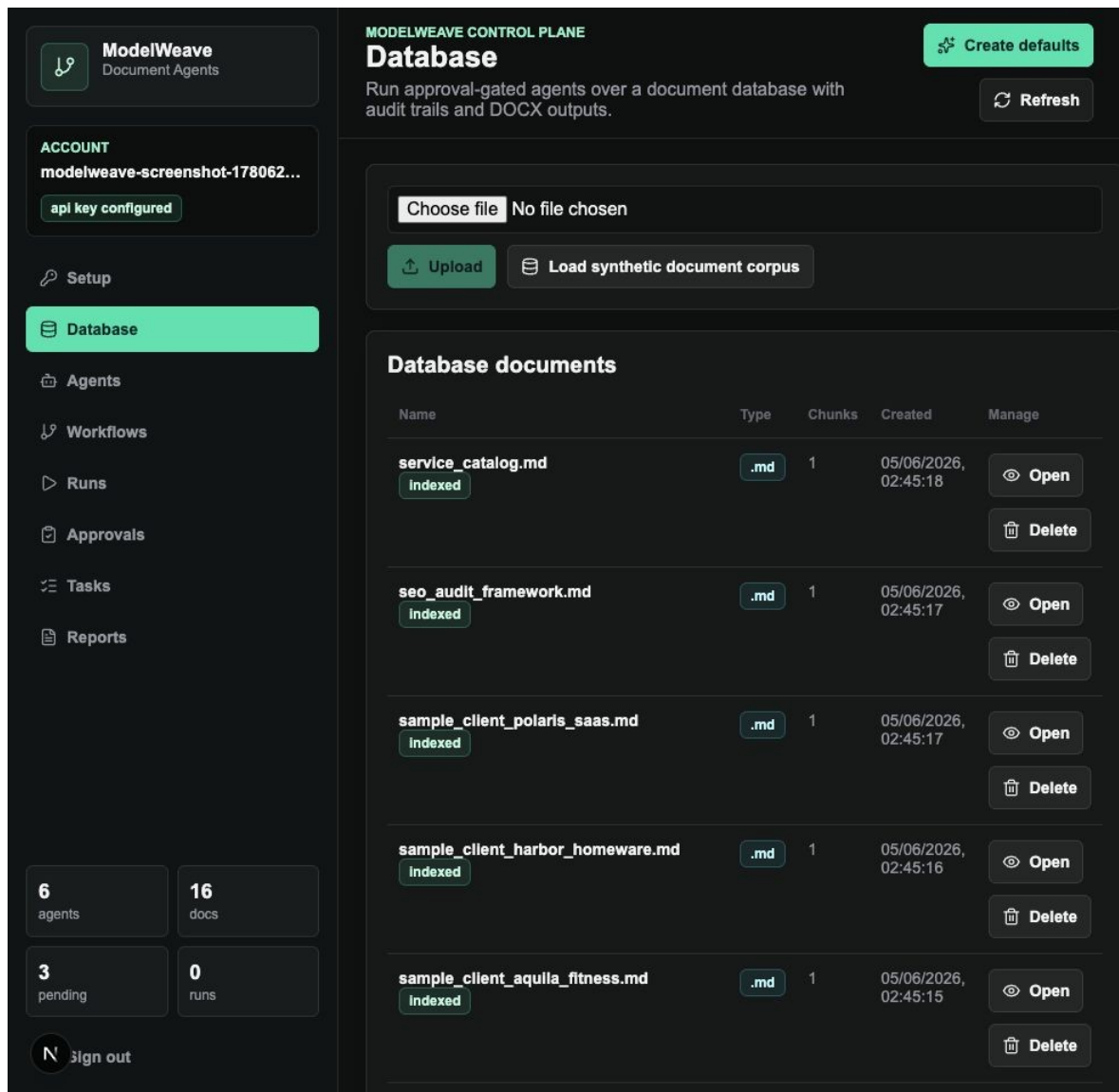


Рисунок 4.3 - Database як база документів

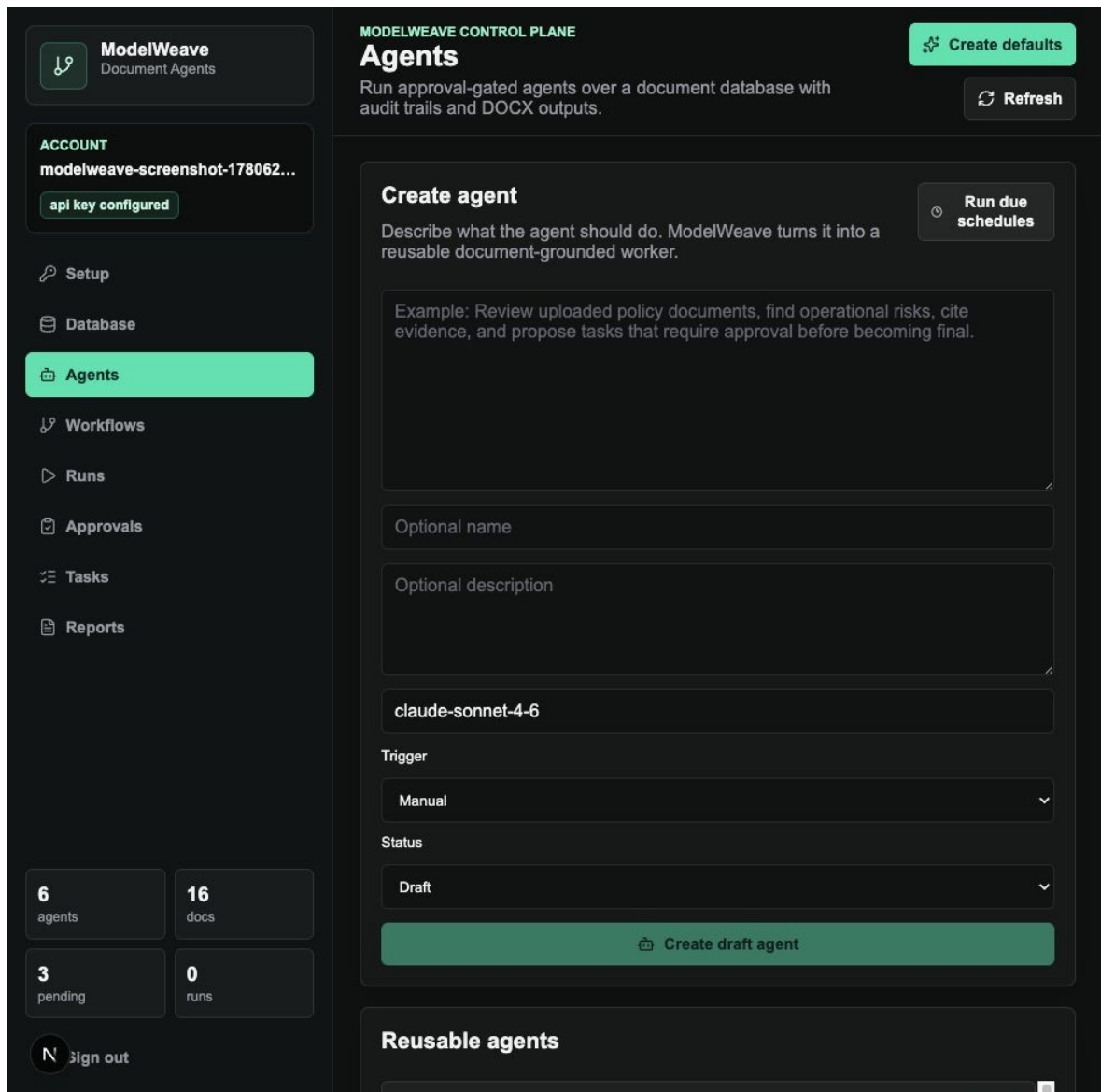


Рисунок 4.4 - Agents та створення документних агентів

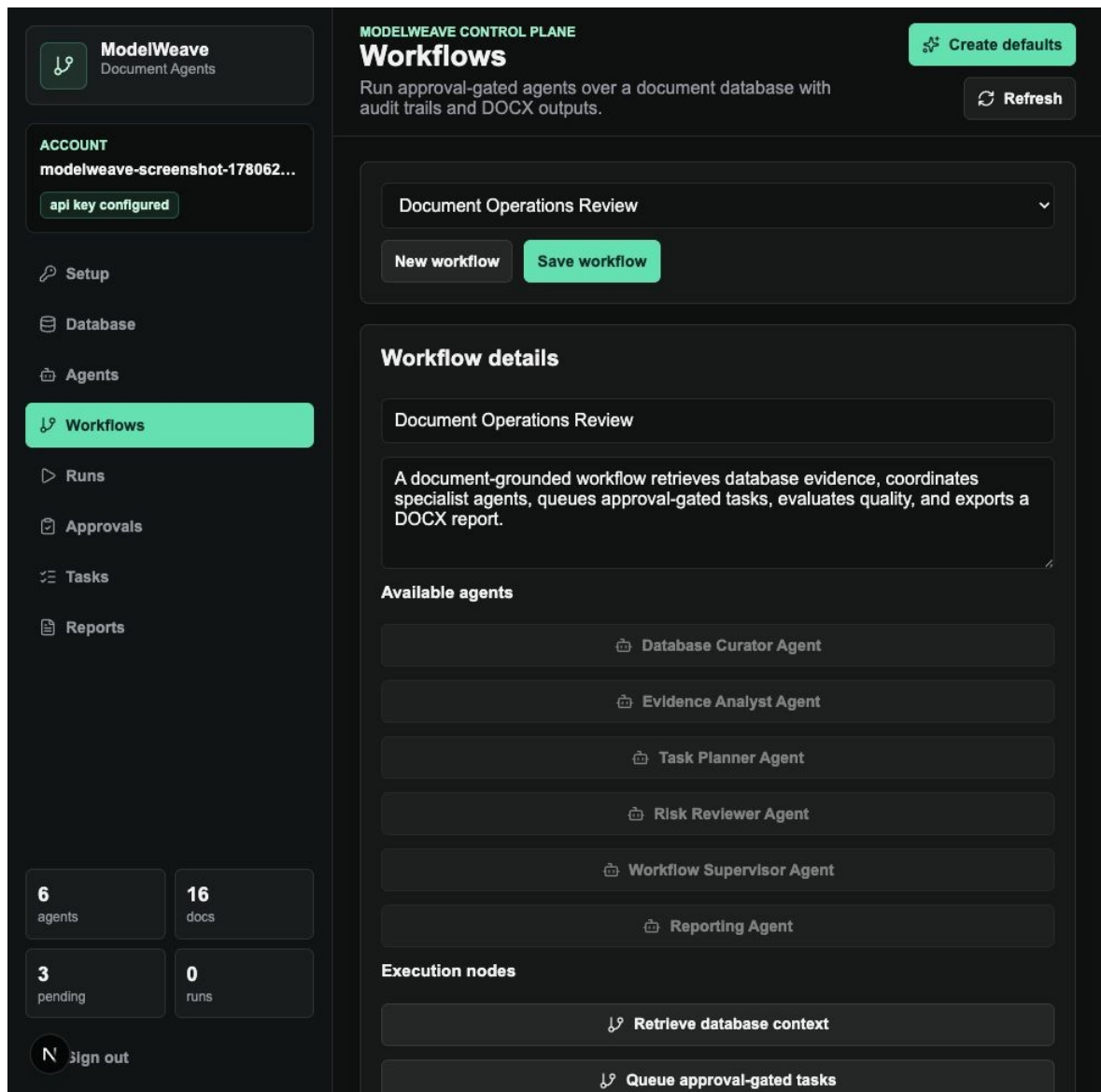


Рисунок 4.5 - Workflows із тунізованими вузлами

4.4 Розгортання

Фронтенд розгорнуто на Vercel. Оновлена production build перевірена 05.06.2026, а свіжий deployment доступний за адресою, наведеною нижче. Бекенд health endpoint Railway також відповідає успішно.

Під час фінальної перевірки було виявлено, що Railway CLI потребує повторного login для примусового backend deploy. Через це production backend health працює, але повне оновлення backend code path має бути виконане після повторної автентифікації Railway. Локальна перевірка backend із реальними Supabase, Qdrant і Anthropic пройшла успішно.

Цей факт важливий для чесної академічної звітності: робота має відокремлювати локально перевірений код, production frontend deployment і

production backend health. Усі ці результати зафіксовано в PROJECT_MEMORY.md та тестових висновках.

Оновлений продукційний фронтенд: <https://modelweave-two.vercel.app>

Продукційний бекенд: <https://api-production-e70a9.up.railway.app>

Публічний репозиторій: <https://github.com/serhiiSotskyi/multi-agent-knowledge-platform>

Висновки до розділу 4

У розділі 4 описано реалізацію ModelWeave. Система побудована як набір сервісів, де UI, API, автентифікація, Postgres, Qdrant, Anthropic і DOCX export мають окремі ролі. Поточний фронтенд узгоджено з новою предметною моделлю Database/Agents/Workflows/Runs/Approvals/Tasks/Reports. Backend локально перевірено end-to-end, а production frontend успішно розгорнуто.

5 ТЕСТУВАННЯ, ОЦІНЮВАННЯ ТА ПРАКТИЧНА ПЕРЕВІРКА

5.1 Методика тестування

Перевірка ModelWeave проводилася на кількох рівнях: статичні перевірки коду, локальні збірки, backend compile, authenticated smoke test, production URL checks, browser UI inspection, screenshot QA, DOCX render QA та незалежний QA subagent audit.

Для backend використано команду `python -m compileall backend/app`. Для frontend використано команду `npm run build`. Для API використано тимчасового підтвердженого Supabase користувача, який створювався перед тестом і видалявся після завершення.

Authenticated smoke test проходив через реальні Supabase Auth, Supabase Postgres, Qdrant Cloud і Anthropic API. Він не був лише mock-тестом. Це важливо, бо саме інтеграції створюють найбільший ризик у сервісно-орієнтованих системах.

UI перевірявся через in-app browser на локальному frontend. Для фінальних скриншотів створено окремого чистого користувача, завантажено synthetic corpus, створено default agents/workflow і виконано один agent run, який сформував approvals, tasks, events і citations.

Таблиця 5.1 - Підсумок фінальних перевірок

Перевірка	Результат	Доказ
Backend compile	Пройдено	backend/.venv/bin/python -m compileall backend/app
Frontend build	Пройдено	npm run build
Local authenticated smoke	Пройдено	31 checks: BYOK, docs, agents, workflow, approvals, tasks, DOCX.
Clean screenshot run	Пройдено	waiting_approval, 3 approvals, 21 events, 8 citations.
Production frontend deploy	Пройдено	Vercel deployment READY.
Production backend health	Пройдено	GET /api/health returned ok.
Production backend new code	Потребує deploy	Railway CLI unauthorized; old defaults still visible in production backend.

5.2 Результати перевірок

Фінальний authenticated smoke test підтвердив, що локальний код виконує повний сценарій. Тимчасовий користувач увійшов через Supabase, зберіг Anthropic key, виконав bootstrap, завантажив synthetic corpus, завантажив і відкрив Markdown-документ, змінив метадані та видалив його.

Тест також створив агента, створив workflow, запустив manual workflow, перевіряв queued-to-running transition, дочекався waiting_approval, перевіряв trace, citations, events і evaluation, прийняв одне затвердження, відхилив інше, перевіряв task statuses і завантажив DOCX report.

Окремо перевірено single-agent run і scheduler endpoint. Scheduled agent створив run із trigger_source='scheduled', що підтверджує єдину історію запусків для ручних і timed-driven процесів.

UI inspection підтвердив наявність усіх потрібних поверхонь: Setup, Database, Agents, Workflows, Runs, Approvals, Tasks і Reports. У Workflows більше немає користувацьких вузлів, пов'язаних із застарілими доменними діями.

DOCX export перевірено через HTTP endpoint. Розмір згенерованого DOCX у smoke test перевищив 40 KB, що підтвердило наявність реального Word-документа, а не порожньої відповіді.



ModelWeave
Document Agents

ACCOUNT
modelweave-screenshot-178062...
api key configured

Setup
Database
Agents
Workflows
Runs
Approvals
Tasks
Reports

6
agents

16
docs

3
pending

0
runs

N sign out

MODELWEAVE CONTROL PLANE
Runs
Run approval-gated agents over a document database with audit trails and DOCX outputs.

Create defaults
Refresh

Manual workflow run
Runs start immediately, then pause only if an approval gate is reached.
Document Operations Review
Review the current document database, identify grounded recommendations, queue approval-gated tasks, and produce a DOCX-ready summary.
Run workflow

Run history

Run	Prompt	Trigger	Status	Inspect
Database Curator Agent 05/06/2026, 02:46:44	Review the current document database and propose approval-gated tasks for improving document-grounded workflo...	manual	waiting_approval	Open

Рисунок 5.1 - Runs із поточним виконанням

43

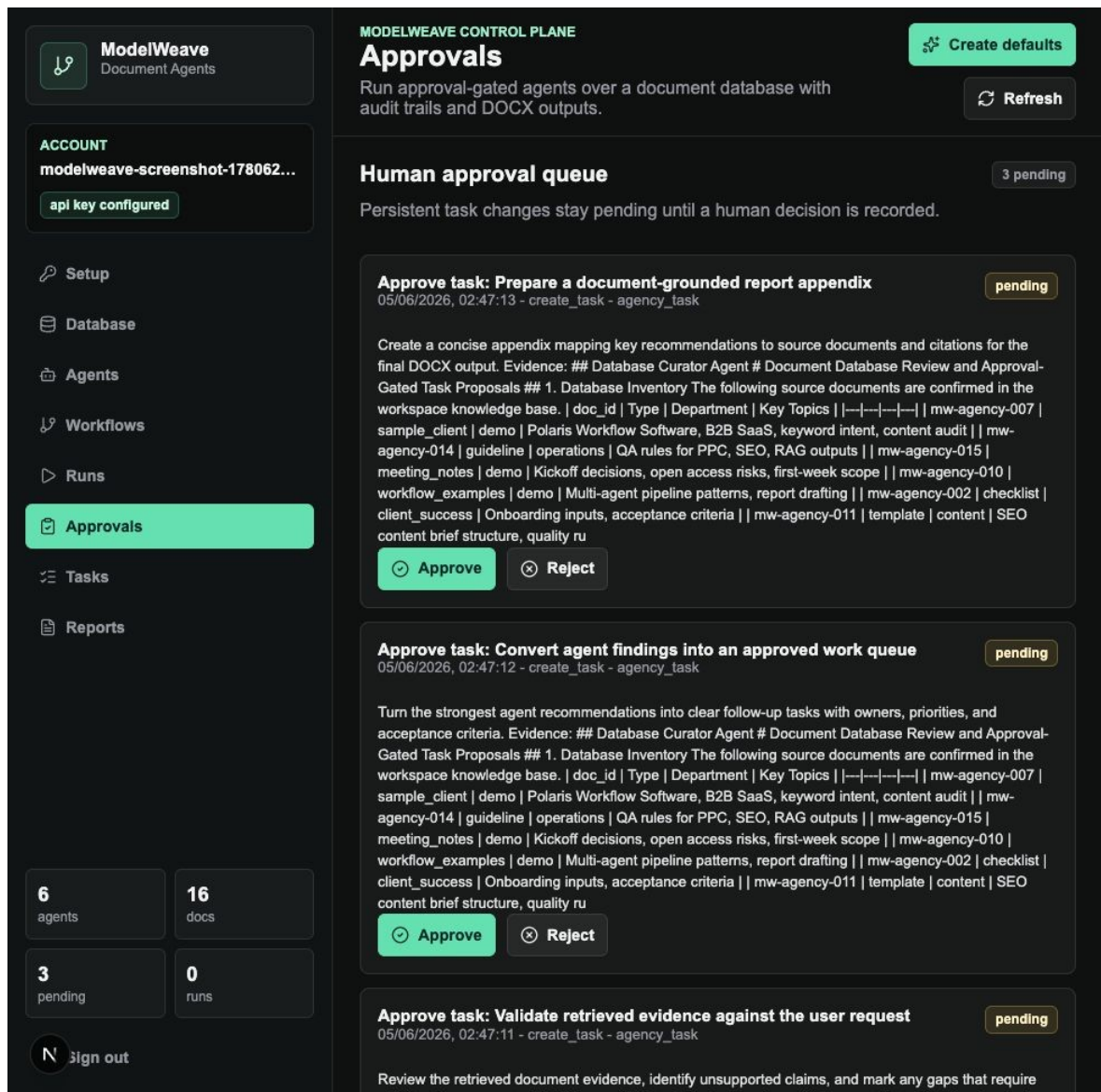


Рисунок 5.2 - Approvals як human gate



ModelWeave
Document Agents

ACCOUNT

modelweave-screenshot-178062...

api key configured

Setup

Database

Agents

Workflows

Runs

Approvals

Tasks

Reports

6 agents

16 docs

3 pending

0 runs

sign out

MODELWEAVE CONTROL PLANE

Tasks

Run approval-gated agents over a document database with audit trails and DOCX outputs.

Create defaults

Refresh

Approved task queue

Tasks are durable follow-up work items created by agents after approval decisions.

Task	Workspace	Discipline	Priority	Status
Prepare a document-grounded report appendix Create a concise appendix mapping key recommendations to source documents and citations for the final DOCX output. Evidence: ## Database Curator Agent # Document Database Review and Approval-Gated Task Proposals ## 1. Database Inventory The following source documents are confirmed in the workspace knowledge base. doc_id Type Department Key Topics --- mw-agency-007 sample_client demo Polaris Workflow Software, B2B SaaS, keyword intent, content audit mw-agency-014 guideline operations QA rules for PPC, SEO, RAG outputs mw-agency-015 meeting_notes demo Kickoff decisions, open access risks, first-week scope mw-agency-010 workflow_examples demo Multi-agent pipeline patterns, report drafting mw-agency-002 checklist client_success Onboarding inputs, acceptance criteria mw-agency-011 template content SEO content brief structure, quality ru	Academic Demo Workspace	reporting	medium	pending_approval
Convert agent findings into an approved work queue	Academic Demo	operations	high	pending_approval

Рисунок 5.3 - Tasks як сталі робочі одиниці



ModelWeave
Document Agents

ACCOUNT
modelweave-screenshot-178062...

api key configured

Setup

Database

Agents

Workflows

Runs

Approvals

Tasks

Reports

6
agents

16
docs

3
pending

0
runs

N

 sign out

MODELWEAVE CONTROL PLANE

Reports
Run approval-gated agents over a document database with audit trails and DOCX outputs.

Create defaults

Refresh

Run history

Report	Status	Score	Created	Actions
Document Database Review and Approval-Gated Task Proposals Database Curator Agent - Review the current document database and propose approval-gated tasks for improving document-grounded workflo...	waiting_approval	0.938	05/06/2026, 02:46:44	Preview DOCX

Report preview

🕒

No report selected
Select Preview to read a formatted report before downloading the DOCX.

Рисунок 5.4 - Reports із DOCX-дією

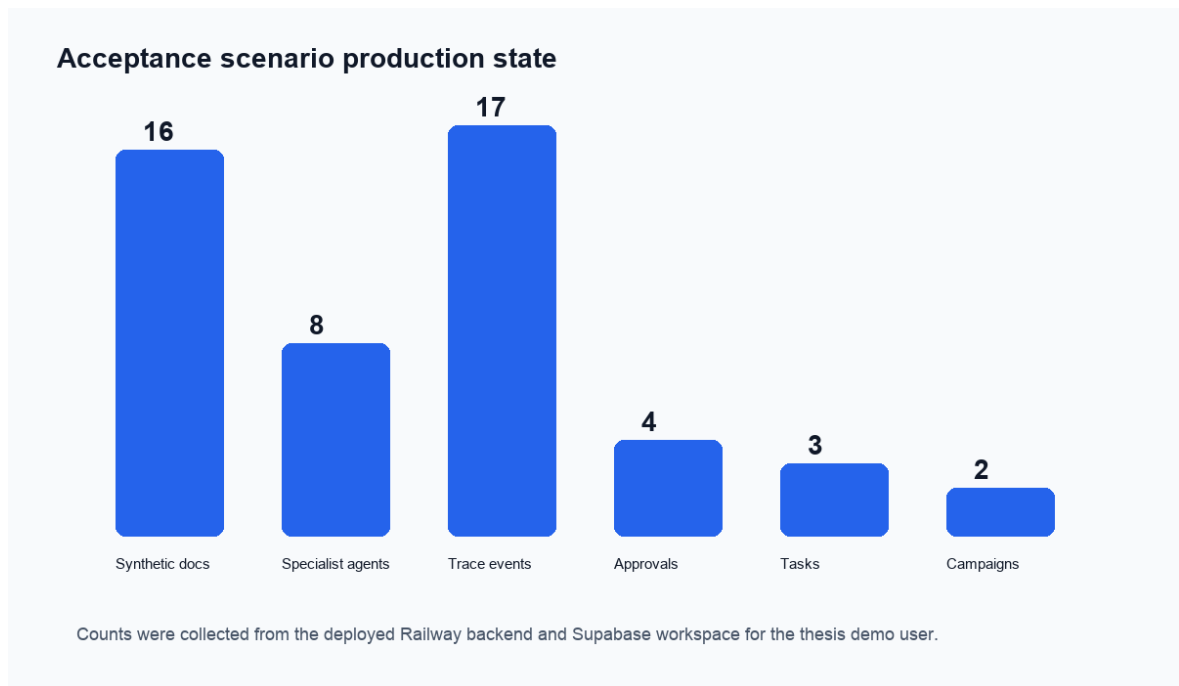


Рисунок 5.5 - Категорії тестових доказів

5.3 Обмеження та напрями розвитку

MVP має обмеження, властиві академічному прототипу. По-перше, дані синтетичні. Це правильне рішення для дипломної роботи, але воно не дає змоги виміряти реальні бізнес-ефекти або виробничі метрики користувачів.

По-друге, background execution реалізовано через FastAPI background tasks. Для продукційної платформи з довгими процесами доцільно використовувати durable queue або workflow engine з повторними спробами, idempotency і спостережуваністю.

По-третє, evaluation score є евристичним. У майбутньому його можна доповнити людськими оцінками, retrieval benchmark, automatic citation checking і regression набором запитів.

По-четверте, scheduled runs реалізовано як lightweight scheduler endpoint. Для production-grade scheduling потрібні cron triggers, locking і захист від дублювання запусків.

По-п'яте, продакційний backend deploy потребує повторної Railway автентифікації. Це не змінює локально перевірену реалізацію, але є операційним пунктом перед остаточною публічною демонстрацією всіх backend features.

Подальший розвиток може включати durable queues, workspace templates, deeper workflow branching, tool permissions, richer evaluation, artifact versioning і зовнішні connectors. Важливо, що ці покращення можуть додаватися як нові сервіси або typed nodes без руйнування основної моделі.

5.4 Оцінка готовності до демонстрації та продукційного використання

Поняття production readiness у межах цієї роботи трактується як готовність системи до контрольованої публічної демонстрації, а не як повна промислова експлуатація з гарантованим SLA. Для академічного прототипу важливо показати, що ключові сценарії працюють, що ризики зафіксовані, а межі відповідальності сервісів зрозумілі.

Перший критерій готовності - відтворюваність. Проєкт має публічний репозиторій, локальні команди перевірки, міграції, production frontend, backend health endpoint і збережені evidence artifacts. Це означає, що результат можна перевірити не лише з тексту диплома, а й через код, скриншоти та тести.

Другий критерій - безпека даних. Реєстрація і сесії реалізовані через Supabase Auth, а агентні запуски використовують BYOK. Це зменшує залежність від одного спільного ключа провайдера і краще відповідає комерційним сценаріям, де користувач має контролювати власні витрати і доступ до моделі.

Третій критерій - контроль дій. Система не дозволяє агентам безпосередньо виконувати сталі бізнес-мутації. Пропозиції проходять approval gate, а rejected approvals залишаються в аудиті. Така поведінка відповідає вимогам відповідального використання LLM у процесах, де текст моделі може бути переконливим, але не повинен автоматично ставати наказом до виконання.

Четвертий критерій - спостережуваність. Runs screen, action_events, trace, citations і evaluation дають змогу пояснити, що відбулося під час запуску. Для демонстрації це критично: користувач бачить не тільки фінальний текст, а й проміжні стани, джерела та рішення.

П'ятий критерій - якість артефактів. Вебзастосунок рендерить Markdown як форматований контент, а DOCX export створює Word-документ із реальними

заголовками, списками, таблицями, оцінками, approval summary і citations. Це робить результат придатним для передачі поза інтерфейсом.

Таблиця 5.2 - Оцінка готовності системи

Критерій	Стан у MVP	Висновок
Відтворюваність	Код, скриншоти, gender QA, тести і production links збережені	Достатньо для академічної перевірки
Безпека	Supabase Auth, BYOK, user-scoped retrieval	Базовий рівень реалізовано
Контроль дій	Approval gate перед сталими змінами	Ризик автономних мутацій знижено
Спостережуваність	Runs, events, trace, citations, evaluation	Запуск можна пояснити
Розгортання	Vercel frontend ready; Railway health OK; backend redeploy потребує login	Є один операційний пункт перед повною production demo
Масштабування	SOA, Qdrant, Postgres, typed nodes	Архітектура готова до поетапного розвитку

З таблиці 5.2 видно, що більшість критеріїв виконано на рівні MVP. Найважливішим відкритим операційним пунктом залишається примусовий redeploy backend на Railway після повторної автентифікації CLI. Це не заперечує реалізацію, але має бути явно зазначено перед демонстрацією, щоб не змішувати локально перевірений backend із попередньою production-версією.

З позиції прикладної математики найбільш значущим результатом є те, що система має формальну основу для масштабування. Збільшення кількості документів впливає на $|C|$ і storage complexity; збільшення кількості агентів впливає на N_calls , latency і probability of success; збільшення кількості approvals впливає на human workload. Ці залежності не приховані в коді, а описані у математичній моделі.

Отже, ModelWeave можна вважати готовим до академічної демонстрації як повностековий прототип сервісно-орієнтованої LLM-платформи. Для промислового впровадження потрібні durable queue, повний production backend redeploy, автоматизований CI, ширший набір retrieval benchmarks, захист від prompt injection і формальний процес керування секретами.

Висновки до розділу 5

У розділі 5 проведено перевірку реалізованої системи. Локальні збірки, backend compile, authenticated smoke test, UI inspection, screenshots і production URL checks підтвердили, що ModelWeave реалізує головний сценарій документно обґрунтованих агентів. Виявлене обмеження production backend deployment пов'язане з автентифікацією Railway CLI, а не з локальним кодом. Результати тестування показують, що архітектура є практичною, перевірною та придатною для подальшого розвитку.

ЗАГАЛЬНІ ВИСНОВКИ

У кваліфікаційній роботі досягнуто поставленої мети: спроектовано, формалізовано, реалізовано та перевірено сервісно-орієнтовану архітектуру для масштабованого використання мовних моделей у комерційних рішеннях.

Теоретичний аналіз показав, що LLM має розглядатися як компонент більшої системи, а не як самостійний продукт. Комерційна придатність вимагає автентифікації, ізоляції даних, retrieval, контролю стану, затверджень, журналу подій і експортованих результатів.

Розроблена математична модель описує корпус документів, фрагментацію, ембедінги, нормалізацію, косинусну подібність, top-k retrieval, RAG-генерацію, типізований workflow graph, agent tuple, state transitions, approval queue, quality score, cost, latency і storage complexity.

Практичним результатом є ModelWeave - вебзастосунок із Database, Agents, Workflows, Runs, Approvals, Tasks і Reports. Система підтримує PDF, TXT, MD і DOCX ingestion, BYOK Anthropic, Qdrant retrieval, editable agents, manual і scheduled runs, approval-gated tasks і DOCX export.

Сервісно-орієнтована реалізація використовує Next.js, FastAPI, Supabase Auth, Supabase Postgres, Qdrant Cloud, Anthropic API, Vercel і Railway. Така декомпозиція відповідає вимогам масштабованості та дозволяє незалежно перевіряти кожний шар.

Тестування підтвердило працездатність локальної інтеграції з реальними Supabase, Qdrant і Anthropic. Оновлений frontend розгорнуто на Vercel; production backend health працює, але примусове оновлення Railway backend потребує повторної автентифікації CLI.

Науково-практичний внесок полягає у демонстрації того, як RAG, agent workflow, approval gates і service-oriented architecture можуть бути поєднані в єдину платформу для контрольованої роботи з мовними моделями.

Розвиток роботи доцільно спрямувати на durable queue, розширені графи процесів, формальне evaluation dataset, інтеграції з зовнішніми сервісами, кращу візуалізацію provenance і автоматичне тестування якості цитувань.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Vaswani, A. et al. Attention Is All You Need. Advances in Neural Information Processing Systems, 2017. URL: <https://arxiv.org/abs/1706.03762>
2. Lewis, P. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS, 2020. URL: <https://arxiv.org/abs/2005.11401>
3. Karpukhin, V. et al. Dense Passage Retrieval for Open-Domain Question Answering. EMNLP, 2020. URL: <https://arxiv.org/abs/2004.04906>
4. Reimers, N.; Gurevych, I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP-IJCNLP, 2019. URL: <https://arxiv.org/abs/1908.10084>
5. Malkov, Y.; Yashunin, D. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. IEEE TPAMI, 2018. URL: <https://arxiv.org/abs/1603.09320>
6. Yao, S. et al. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR, 2023. URL: <https://arxiv.org/abs/2210.03629>
7. Shinn, N. et al. Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS, 2023. URL: <https://arxiv.org/abs/2303.11366>
8. Wu, Q. et al. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. Microsoft Research, 2023. URL: <https://arxiv.org/abs/2308.08155>
9. Wang, L. et al. A Survey on Large Language Model Based Autonomous Agents. Frontiers of Computer Science, 2024. URL: <https://arxiv.org/abs/2308.11432>
10. Zhao, W. X. et al. A Survey of Large Language Models. arXiv, 2023. URL: <https://arxiv.org/abs/2303.18223>
11. Robertson, S.; Zaragoza, H. The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in Information Retrieval, 2009. URL: <https://www.nowpublishers.com/article/Details/INR-019>
12. Manning, C.; Raghavan, P.; Schütze, H. Introduction to Information Retrieval. Cambridge University Press, 2008. URL: <https://nlp.stanford.edu/IR-book/>
13. Anthropic Claude API documentation. Official documentation, accessed 2026. URL: <https://docs.anthropic.com/>
14. Anthropic Messages API. Official documentation, accessed 2026. URL: <https://docs.anthropic.com/en/api/messages>
15. OWASP Foundation OWASP Top 10 for Large Language Model Applications. Official project, 2025. URL: <https://owasp.org/www-project-top-10-for-large-language-model-applications/>
16. NIST Artificial Intelligence Risk Management Framework (AI RMF 1.0). National Institute of Standards and Technology, 2023. URL: <https://www.nist.gov/itl/ai-risk-management-framework>

17. Supabase Authentication documentation. Official documentation, accessed 2026. URL: <https://supabase.com/docs/guides/auth>
18. Supabase Row Level Security documentation. Official documentation, accessed 2026. URL: <https://supabase.com/docs/guides/database/postgres/row-level-security>
19. Qdrant Qdrant vector database documentation. Official documentation, accessed 2026. URL: <https://qdrant.tech/documentation/>
20. Qdrant Collections and points documentation. Official documentation, accessed 2026. URL: <https://qdrant.tech/documentation/concepts/collections/>
21. FastAPI FastAPI documentation. Official documentation, accessed 2026. URL: <https://fastapi.tiangolo.com/>
22. Next.js App Router documentation. Official documentation, accessed 2026. URL: <https://nextjs.org/docs/app>
23. Vercel Deployments documentation. Official documentation, accessed 2026. URL: <https://vercel.com/docs/deployments>
24. Railway Railway documentation. Official documentation, accessed 2026. URL: <https://docs.railway.com/>
25. React Flow React Flow documentation. Official documentation, accessed 2026. URL: <https://reactflow.dev/>
26. LangChain LangGraph Platform generally available. Official blog, 2025. URL: <https://www.langchain.com/blog/langgraph-platform-ga>
27. CrewAI CrewAI AMP documentation. Official documentation, accessed 2026. URL: <https://docs.crewai.com/en/enterprise/introduction>
28. Relevance AI Build an AI workforce. Official documentation, accessed 2026. URL: <https://relevanceai.mintlify.dev/docs/build/workforces/build-an-ai-workforce/add-agents>
29. Y Combinator Gumloop company profile. YC profile, accessed 2026. URL: <https://www.ycombinator.com/companies/gumloop>
30. PostgreSQL Global Development Group PostgreSQL Row Security Policies. Official documentation, accessed 2026. URL: <https://www.postgresql.org/docs/current/ddl-rowsecurity.html>
31. Python Software Foundation Python documentation. Official documentation, accessed 2026. URL: <https://docs.python.org/3/>
32. ECMA International Office Open XML File Formats. Standard overview, accessed 2026. URL: <https://ecma-international.org/publications-and-standards/standards/ecma-376/>

ДОДАТКИ

Додаток А. Продукційні посилання

Frontend: <https://modelweave-two.vercel.app>

Backend: <https://api-production-e70a9.up.railway.app>

Repository: <https://github.com/serhiiSotskyi/multi-agent-knowledge-platform>

Додаток Б. Додаткові скриншоти інтерфейсу

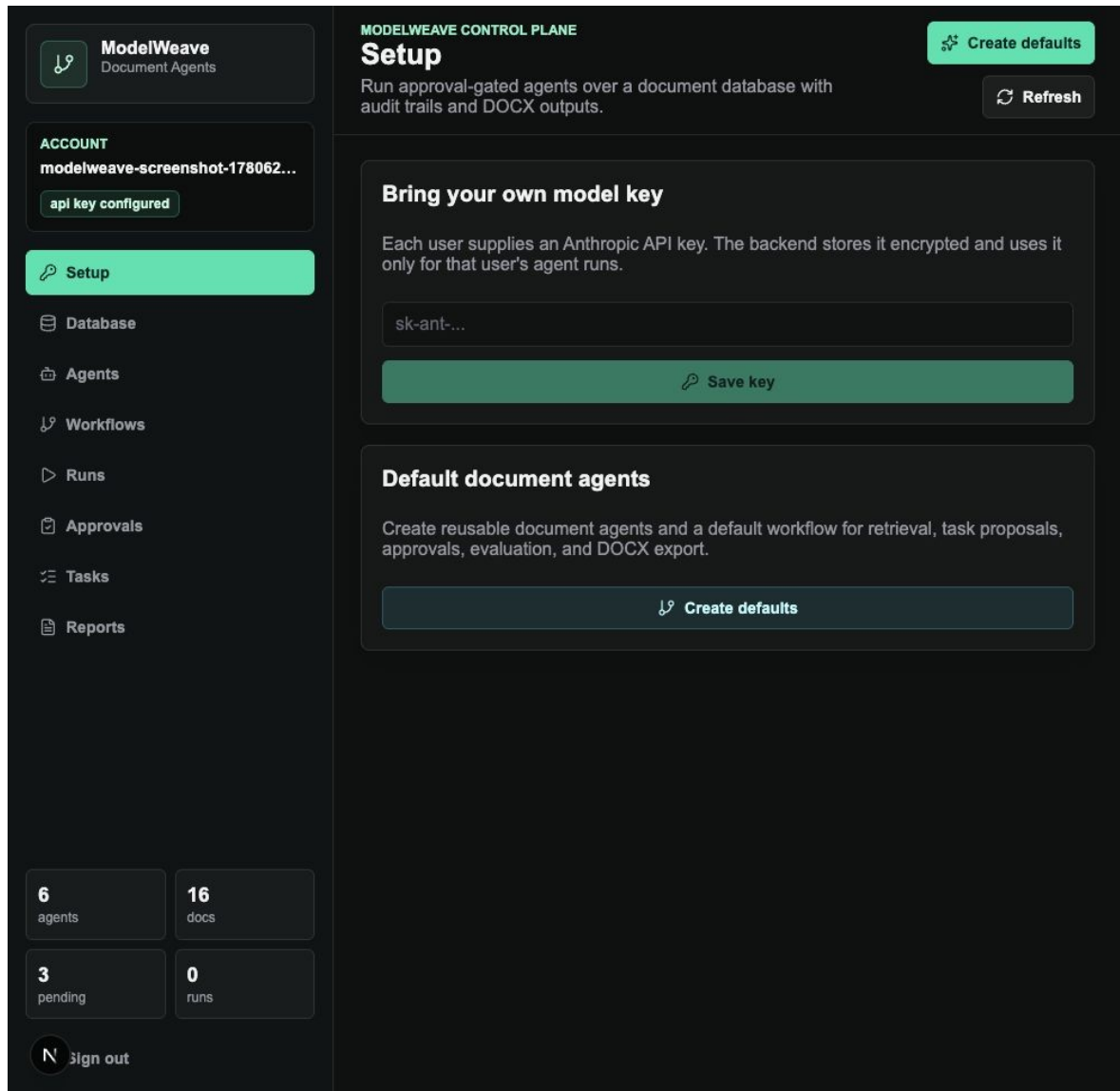


Рисунок Б.1 - Setup

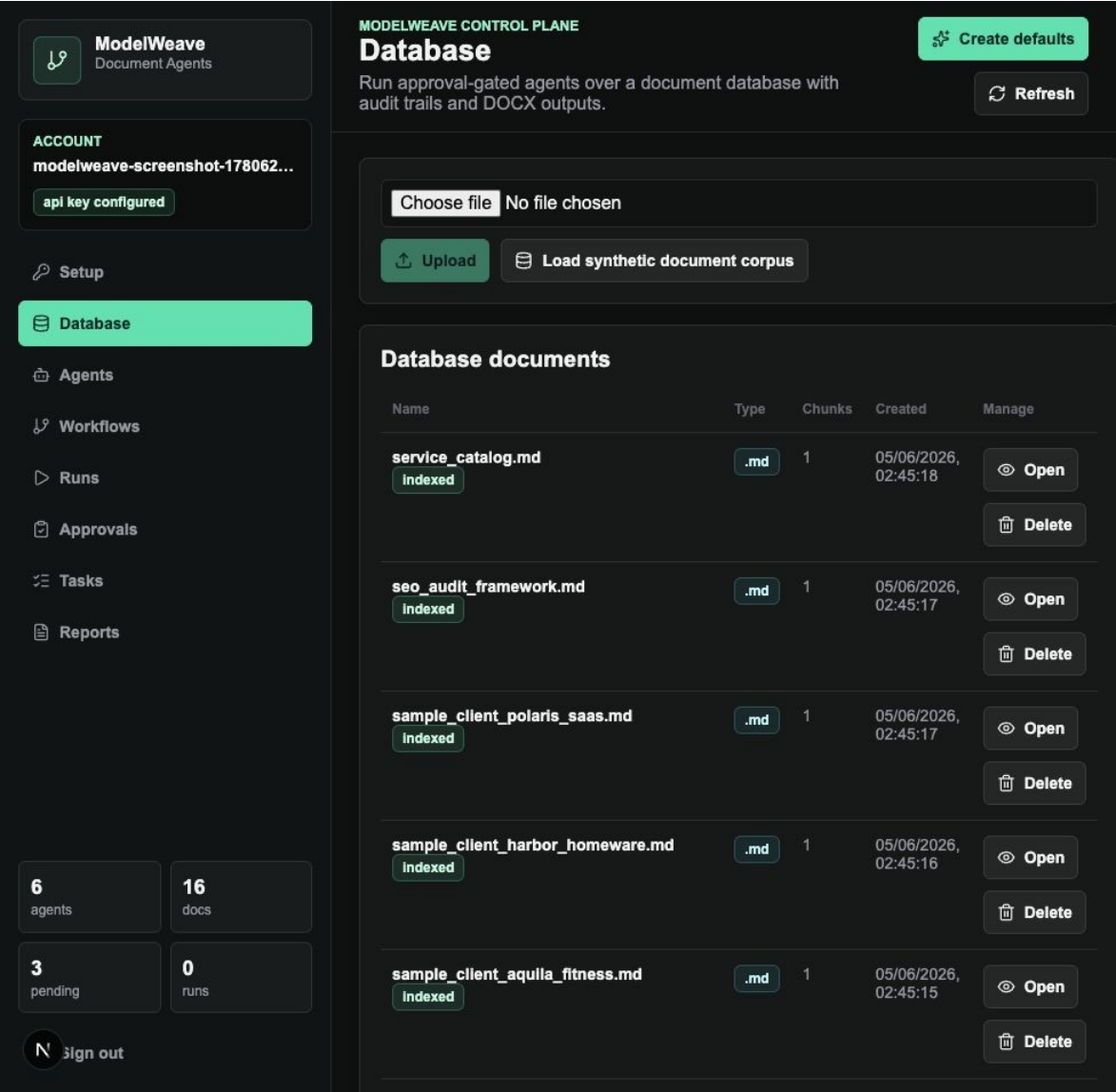


Рисунок Б.2 - Database

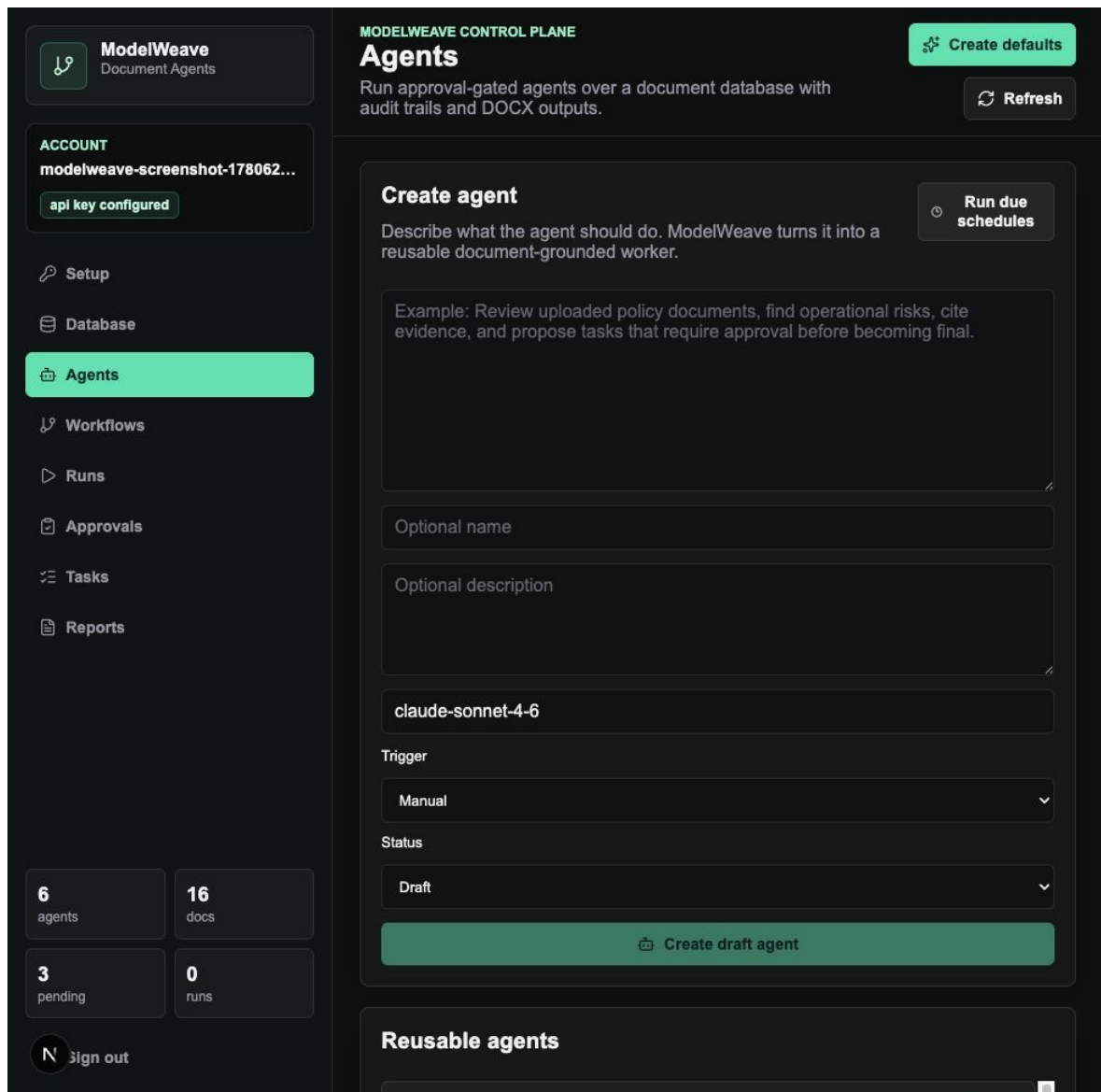


Рисунок Б.3 - Agents

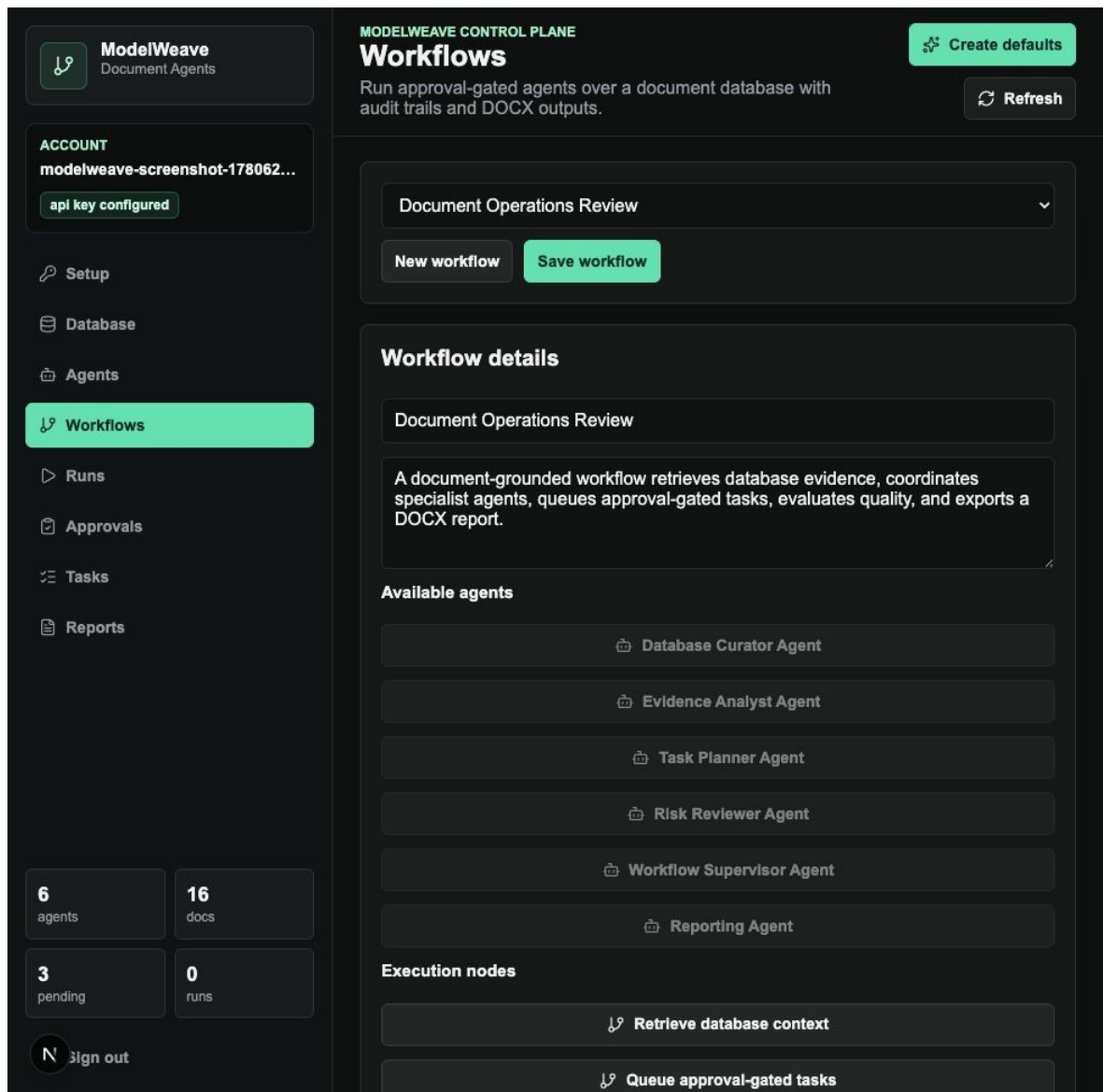


Рисунок Б.4 - Workflows

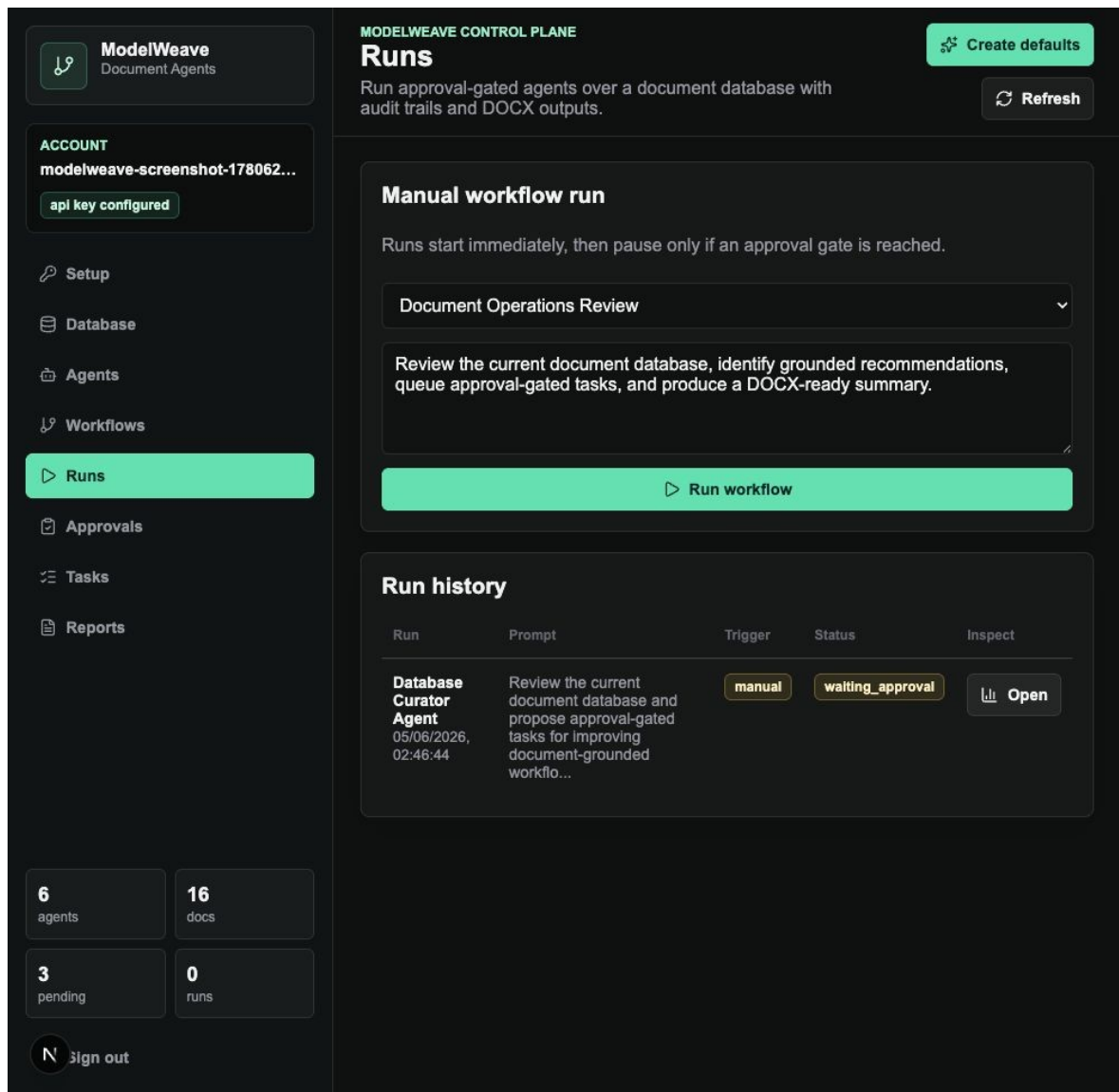


Рисунок Б.5 - Runs

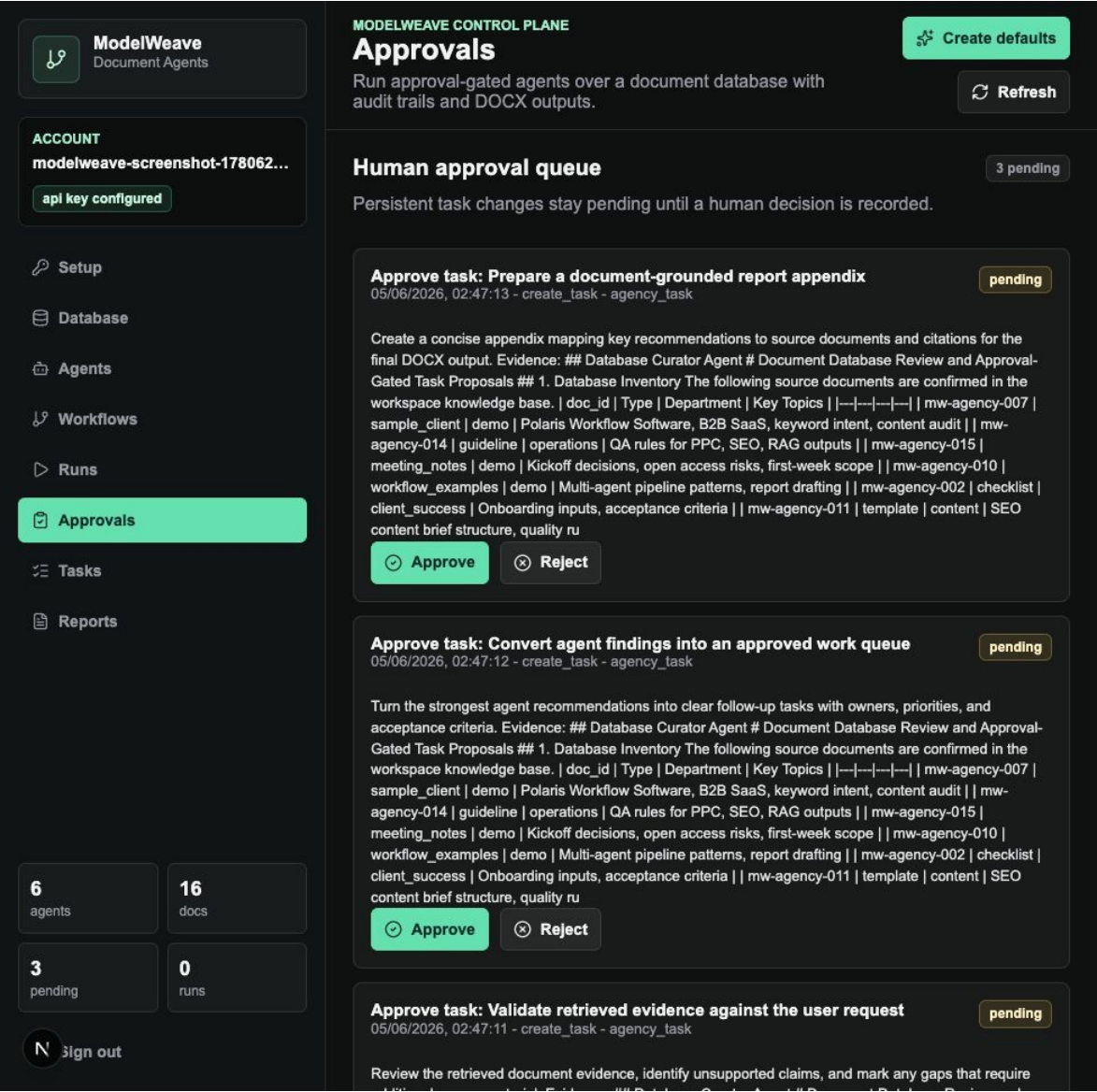


Рисунок Б.6 - Approvals



ModelWeave
Document Agents

ACCOUNT
modelweave-screenshot-178062...
api key configured

Setup

Database

Agents

Workflows

Runs

Approvals

Tasks

Reports

6
agents

16
docs

3
pending

0
runs

 sign out

MODELWEAVE CONTROL PLANE

Tasks

Run approval-gated agents over a document database with audit trails and DOCX outputs.

Create defaults

Refresh

Approved task queue

Tasks are durable follow-up work items created by agents after approval decisions.

Task	Workspace	Discipline	Priority	Status
Prepare a document-grounded report appendix Create a concise appendix mapping key recommendations to source documents and citations for the final DOCX output. Evidence: ## Database Curator Agent # Document Database Review and Approval-Gated Task Proposals ## 1. Database Inventory The following source documents are confirmed in the workspace knowledge base. doc_id Type Department Key Topics --- mw-agency-007 sample_client demo Polaris Workflow Software, B2B SaaS, keyword intent, content audit mw-agency-014 guideline operations QA rules for PPC, SEO, RAG outputs mw-agency-015 meeting_notes demo Kickoff decisions, open access risks, first-week scope mw-agency-010 workflow_examples demo Multi-agent pipeline patterns, report drafting mw-agency-002 checklist client_success Onboarding inputs, acceptance criteria mw-agency-011 template content SEO content brief structure, quality ru	Academic Demo Workspace	reporting	medium	pending_approval
Convert agent findings into an approved work queue	Academic Demo	operations	high	pending_approval

Рисунок Б.7 - Tasks

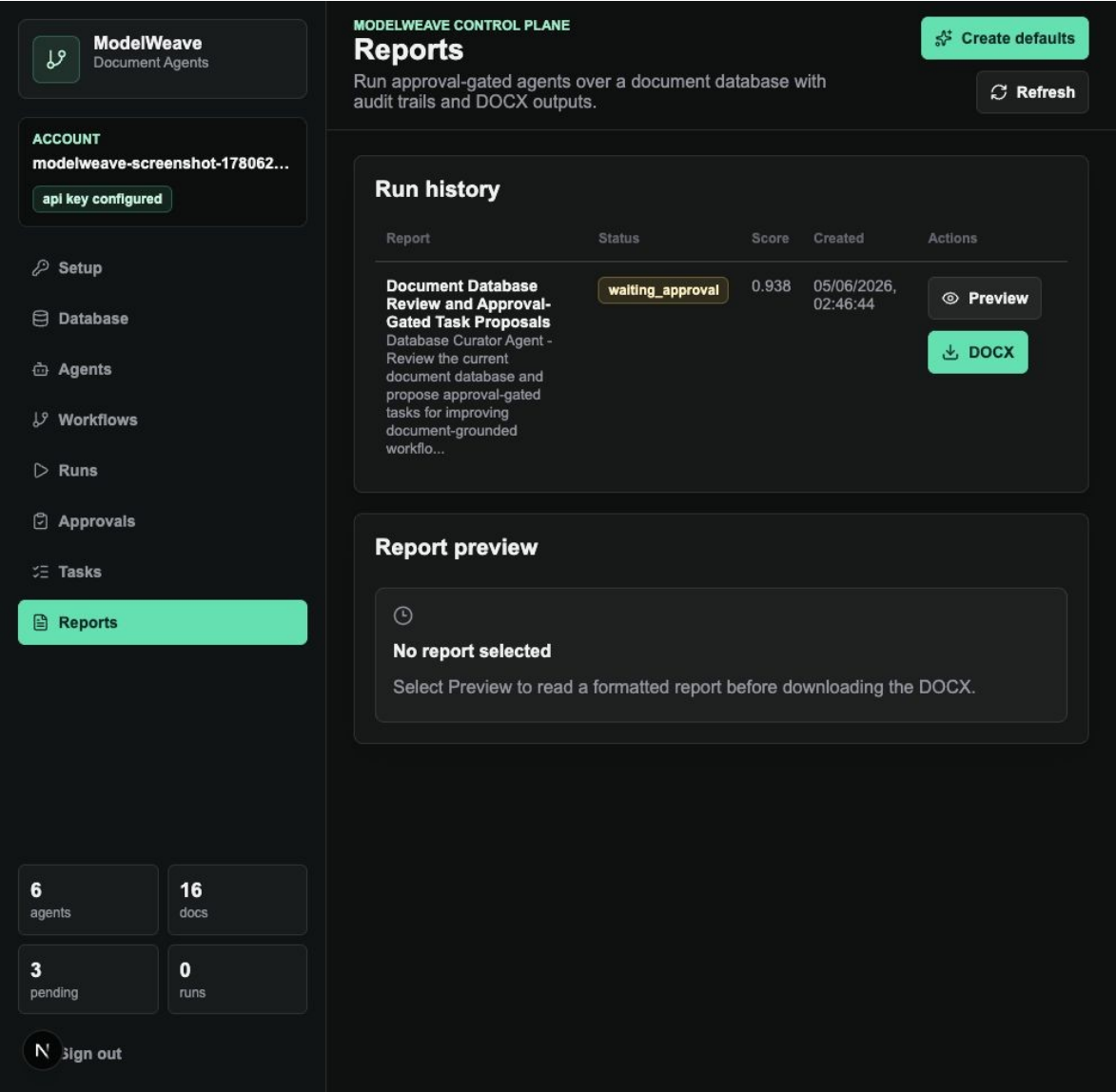


Рисунок Б.8 - Reports

Додаток В. API-ресурси

Таблиця В.1 - API поточної системи

Ресурс	Маршрути
Database	GET/POST/PUT/DELETE /api/documents; POST /api/documents/seed-synthetic
Agents	GET/POST/PUT /api/agents; POST /api/agents/{id}/run
Scheduler	POST /api/agents/scheduler/run-due
Workflows	GET/POST/PUT /api/workflows
Runs	POST /api/runs; GET /api/runs; GET /api/runs/{id}
Approvals	GET /api/approvals; POST /api/approvals/{id}/approve; POST /api/approvals/{id}/reject
Tasks	GET/POST/PUT /api/tasks
Reports	GET /api/runs/{id}/docx

Додаток Г. Заява щодо даних

У роботі не використовуються реальні корпоративні дані. Синтетичний корпус створено для демонстрації завантаження, фрагментації, vector retrieval, агентних запусків, затверджень і DOCX-експорту. Тимчасові користувачі, створені під час QA, були видалені після перевірок.